Monitoring UK 2040 Agenda Transport zero-emission Using R-shiny

Author: Eduard De Jesus Romero Mercedes

Student ID: 20041621

Course: MSc Data Analytics summer 2022

Supervisor: Professor Hassan Kazemian

ABSTRACT

This work is aimed at integrating all the relevant information related to the UK 2040 Zero Emissions Agenda into an App. The project is an interactive application that visualises the objective's status and monitors its progress through R shiny mapping the results geographically and graphically.

CONTENT

Abstract 1

List of figures 2

Acknowledgements 4

Chapter 1: Introduction 5

Chapter 2: a review of literature 5
- Evolution of data visualization 5
- R Programming 8
- R shiny 10
- leaflet 11

Chapter 3: methodology 12
- Introduction 12
- Defining the goal 14
- Gathering data 16
- Cleaning Data 17
- Creating the R-shiny template 18
- Developing The Visual Structure 19
- Deploying the model 21

Chapter 4: Implementation 22
- introduction 22
- Gathering Data 22
- cleaning data 29
- creating the R shiny template 34

Chapter 5: testing-results 47

Chapter 6: DISCUSSION CONCLUSION 48

Bibliographies 49

appendices 50
- code 50

FIGURE 1 CHART OF HISTORY

FIGURE 2 MINARD JOSEPH. NAPOLEON'S RUSSIAN CAMPAGNE

FIGURE 3 MICROSOFT POWERBI

FIGURE 4 FLASK DASHBOARD

FIGURE 5 R STRUCTURE (own production

FIGURE 6. HADLEY WICKHAM HTTPS://HADLEY.NZ/

FIGURE 7. SHINY STRUCTURE

FIGURE 8.R SHINY LOGO HTTPS://WWW.RSTUDIO.COM/

FIGURE 9. Data analysis process HTTPS://WWW.G2.COM/ARTICLES/DATA-ANALYSIS-PROCESS

FIGURE 10 DATA RELATIONSHIP (own production)

FIGURE 11 ELECTRICITY SUPPLY BY REGION (OWN PRODUCTION)

FIGURE 12 ELECTRICITY BY REGION TRANSFORMED (OWN PRODUCTION)

FIGURE 12. R FOR DATA SCIENCE. HADLEY WICKHAM HTTPS://R4DS.HAD.CO.NZ/

FIGURE 13 R SHINY TEMPLATE (OWN PRODUCTION)

FIGURE 14 WASHINGTON POST ARTICLE
HTTPS://WWW.WASHINGTONPOST.COM/SF/LOCAL/2013/11/09/WASHINGTON-A-WORLD-APART

FIGURE 15 BARRET SCHLOERKE : HTTPS://SHINY.RSTUDIO.COM/GALLERY/SUPERZIP-EXAMPLE.HTML

FIGURE 16. EDWARD PARKERHTTPS://SHINY.RSTUDIO.COM/GALLERY/COVID19-TRACKER.HTML

FIGURE 17 RSTUDIO SERVER SHINYIO.RSTUDIO.COM

FIGURE 18 HTTPS://BORDERS.UKDATASERVICE.AC.UK/BDS.HTML

FIGURE 19 INFUSE_CTRY_2011.DBF(own production)

FIGURE 21 SUBNATIONAL_ELECTRICITY_CONSUMPTION_STATISTICS_2005-2020.XLSX (OWN PRODUCTION)

FIGURE 22 GREENHOUSE GAS EMISSION DATA (own production)

FIGURE 23
HTTPS://WWW.DATA.GOV.UK/DATASET/723C243D-2F1A-4D27-8B61-CDB93E5B10FF/UK-LOCAL-AUTHORITY-AND-REGIONAL-GREENHOUSE-GAS-EMISSIONS

FIGURE 24 CHAGER POINTS (own production)

FIGURE 25 ELECTRIC VEHICLE DATASET (own dataset)

FIGURE 26 AT HTTPS://WWW.GOV.UK/GOVERNMENT/STATISTICAL-DATA-SETS/VEHICLE-LICENSING-STATISTICS-DATA-TABLES

FIGURE 27 GEOCODE LABELS (own production)

FIGURE 28 ELETRICITY CONSUMPTION BY REGION

FIGURE 29 ELECTRICITY CONSUMPTION (own production)

FIGURE 30 GREENHOUSE EMISSION (own production)

FIGURE 31 HTTPS://GEOPORTAL.STATISTICS.GOV.UK/DATASETS/7606BABA633D4BBCA3F2510AB78ACF61/ABOUT

FIGURE 32 TEMPLATE (own production)

FIGURE 33 PLOTS TAB (own production)

FIGURE 34 ADDTILES OBJECT (own production)

FIGURE 35 ADDING LAYERS (own production)

FIGURE 36 CHARGE POINTS (own production)

FIGURE 37 CO2 EMISSION BY COLOR (own production)

FIGURE 38 SAMPLE MAP (own production)

FIGURE 39 SAMPLE OF PLOT TAB (own production)

FIGURE 40 PUBLISHING THE APP(OWN PRODUCTION)

FIGURE 41 SERVER (own production)

FIGURE 42 HAVING ACESS (OWN PRODUCTION)

To Smilena, my support and my moral guidance in my difficult situations and Professor Hassan Kazemian for all his support.

The last decades have been marked by the development of multiple tools that allow us to visualize data, develop models and at the same time interact in real-time with those data, creating some routines to manipulate certain information and generate predictions. Some examples are Tableau, Powerbi, Qlik, R shiny or Flask from python.

The difference with the traditional methodology lies in user interaction as we can now perform some operations, navigate between different data detail levels, or navigate to a different point on a map. All these possibilities require mastering more sophisticated skills but in some cases like Tableu or Powerbi the coding skills are not highly required, in contrast, for others tools like Flask or R shiny the coding skill is a fundamental factor.

In this work, we will explore some of the solutions that R offer to the user to manipulate and represent data geographically and interactively.

EVOLUTION OF DATA VISUALIZATION

In 1769 Joseph Priestly published **A new Chart of History** a very impressive illustration that sums up in a chart more than 2000 thousand years of history which list events in 106 location and names the most important characters in history like Cleopatra, Alexander or some biblical character like Salomon.

This chart is considered one of the first timeline charts in history and even today we can appreciate the effort required to produce such a work.

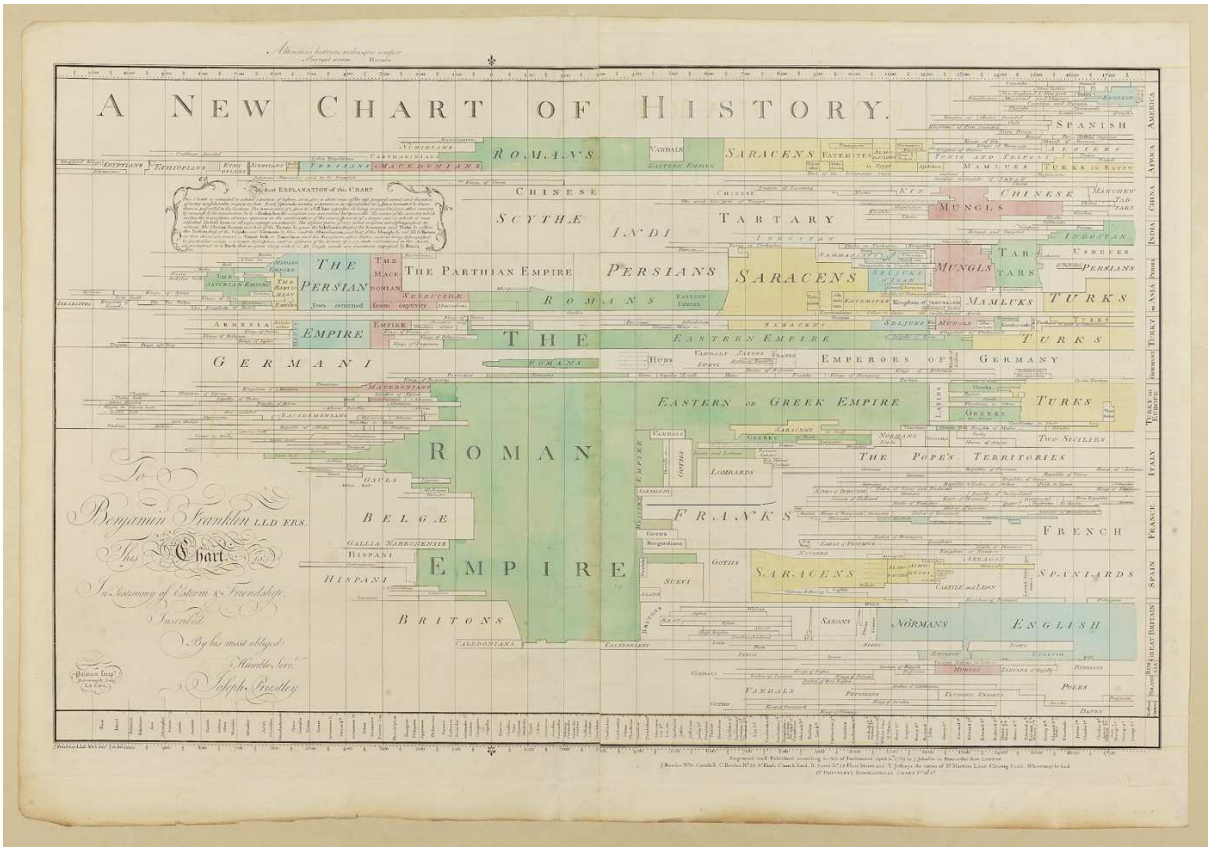


FIGURE 1 CHART OF HISTORY HTTPS://EN.WIKIPEDIA.ORG/WIKI/JOSEPH_PRIESTLEY

In figure 1, we can see a timeline on one of the axes and the name of several regions on the other. The intention of this plot was to represent the influence of distinct empires and cultures over other regions. Although the regions are not represented geographically, it does refer to them in an abstract way. The reason why in this case the region wasn't represented physically is that it could take pretty much work since for each year it should represent a different map

But in 1869 Charles Joshep Minard published **Carte figurative des pertes successives en hommes de l'Armée Française dans la campagne de Russie 1812-1813** an illustration that summarizes six sets of data in a single graph considering the temperature, the situation and the direction of the troops, movement of division and regrouping, as well as the decline of the army. In one single illustration, Charles managed to represent several variables in a very meaningful way representing not only the time but the geographic location. But this time the time was represented by colour and direction

over the same map. Each colour represents the direction, and the time was represented by labels which is an improvement since we can ask at what point was the army in a determined period of time. So, we see a pattern emerging when we want to represent time and space in a single graph. If we want to represent different variables in 2D dimension these variables need to be represented by colour and location. It means one single location can show a different period of time if the colour is different

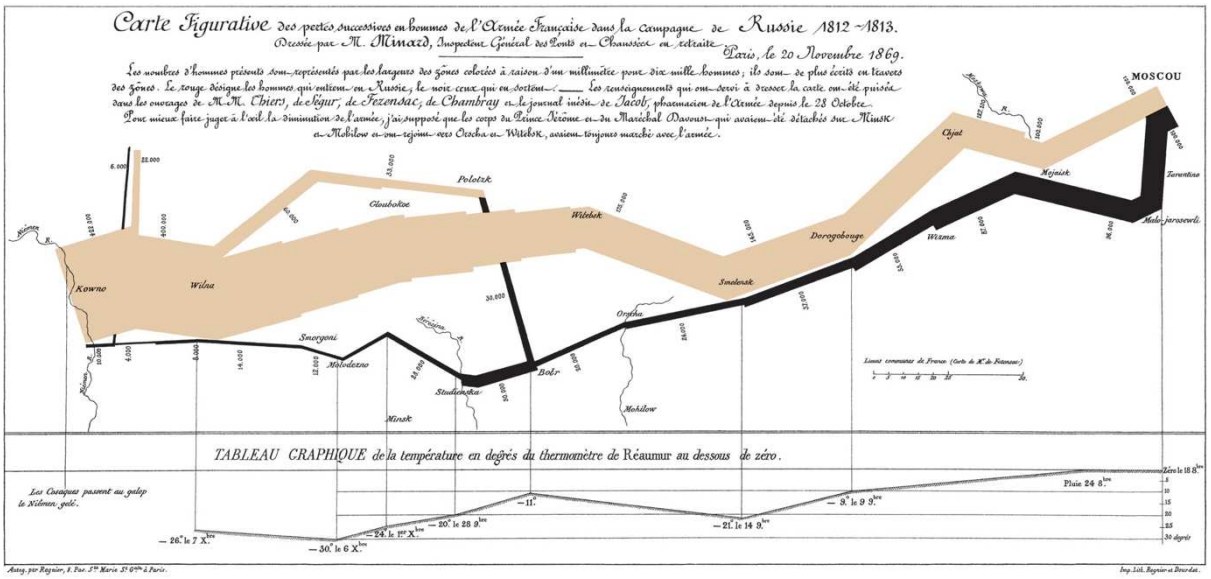


FIGURE 2 MINARD JOSEPH. NAPOLEON'S RUSSIAN CAMPAGNE HTTPS://EN.WIKIPEDIA.ORG/WIKI/FILE:MINARD.PNG

Making a huge leap in history in the early 1990s, Microsoft released Microsoft Excel. A software capable of storing and organizing unstructured data and then visualizing it. We can say that this was the first game changer in the data visualization industry, as all the visualizations were mainly oriented toward scientific topics and for the first time, other non-scientific profiles could access data manipulation and visualization. Later, in 2011, after all the experience gained in working with data, Microsoft launched PowerBi, one of the first data visualization tools with a friendly interface for broad users and non-scientific purposes, unlike most of the other tools back then that required a highly trained staff to manipulate and plot data. For the first time, different types of operations could be carried out in a single tool inside the same environment. In this case, power bi offers a wide selection of data manipulation and modelling in the Kimball scheme and build a smart application for business.



FIGURE 3 MICROSOFT POWERBI HTTPS://POWERBI.MICROSOFT.COM/EN-GB/

At the same time, other tools flooded the market like Tableau or Qlik and then we started talking about a new role or the person who would be in charge of managing these new tools to generate value for the company using data to summarize the most important hidden keys in the data and communicate it through graphical objects or mathematical models.

Although, despite all the effort to provide tools to easily work with data, there are still some tools aimed at more technical profiles to create more elaborate products to solve sophisticated problems such as Flask or R-shiny. These tools are based on web technologies to take advantage of all the functionalities that this field provides to the end user, such as creating a friendly and customizable user interface or a very impressive application that can also generate some results, such as loading data sets and automatically selecting columns and plot it against each other. The possibilities that everyone can realize are endless only limited by programming skills. The disadvantage of using these tools is the high complexity that in some cases is worthless if the application is not trying to develop a product that is easily achieved by other premade tools like Powerbi or Tableau. The use of tools like R shiny is recommended when the application is aimed to develop an application that requires a certain type of modelling that other tools do not support, or the customer requires high customization in its application. All these requirements can be fulfilled by tools like flask or R-shiny

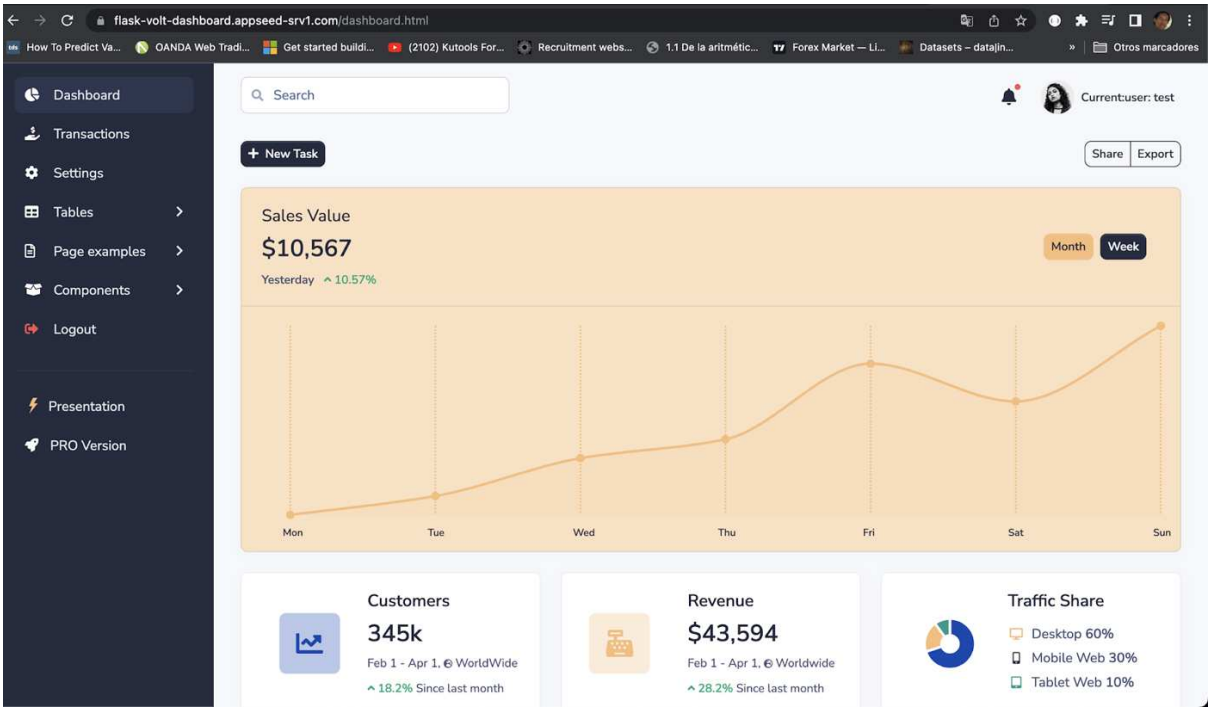


FIGURE 4 FLASK DASHBOARD HTTPS://FLASK-VOLT-DASHBOARD.APPSEED-SRV1.COM/DASHBOARD.HTML

R PROGRAMMING

R is a programming language for scientific computing overall statistics and machine learning supported by the R Core Team and the R Foundation for Statistical Computing. Created by Ross Ihaka and Robert Gentleman.

The official R software environment is an open-source free software environment within the GNU package, available under the GNU General Public License. R is primarily written in C, Fortran, and R itself.

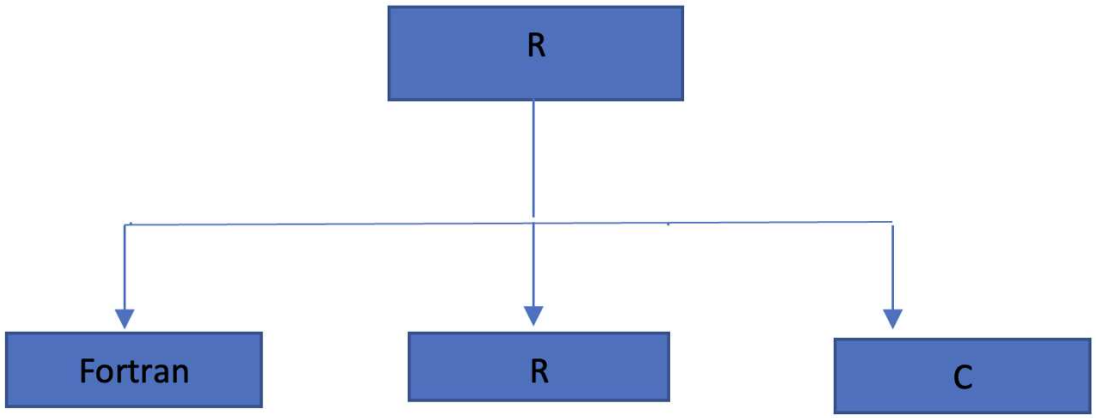


FIGURE 5 R STRUCTURE (OWN PRODUCTION)

At its start, R was an implementation of the S programming language combined with lexical scoping semantics from “scheme”, which allows objects to be defined in predetermined blocks rather than the entirety of the code.

R’s data structure includes vectors, arrays, lists, and data frames. The latter is one of the most successful data structures in R since it is one of the most used during data processing and data manipulation allowing different operations. Recently, other structures were added as the third party like Tibble which is similar to a data frame with some modifications that allow speed up some transformation.

Tibble data structure belongs to Tidyverse Package. Tidyverse Package is a collection of open-source packages for the R programming language introduced by Hadley Wickham and his team that share an underlying design philosophy, grammar, and data structures called tidy data. According to Hadley Wickham, data is considered tidy if:

1. Each variable must have its own column
2. Each observation must have its own row
3. Each value must have its own cell



Figure 6. Hadley Wickham https://hadley.nz/

The previous philosophy is achieved thanks to numerous packages in the tidyverse package like Dplyr or Forcat which we will be using in this work

In November 2012 Rstudio released Shiny, a new package for R. It allowed R developers to build simple interactive Web-based interfaces for R scripts, using only R code. Shiny is an R package that makes it easy to build interactive web applications directly from R. These capabilities can be extended using CSS themes, HTML widgets, and JavaScript actions. This tool has been gaining popularity due to its high customization possibilities and powerful tools. Unlike other tools like Tableau or Powerbi, shiny R allows you to apply a wide selection of statistical models in real-time and check the results only limited by the variety of packages other people bring to R, and it keeps growing.

The structure of the R-shiny document is composed of Three parts:

- UI: User Interface
- Server: The backend area code
- Shinyapp(ui,server): The execution area code

```
library(shiny)

ui <- fluidPage()

server <- function(input, output){}

shinyApp(ui = ui, server = server)
```

FIGURE 7. SHINY STRUCTURE (OWN PRODUCTION)

In the UI section, we write all the code necessary for the visual interaction of the user with the application. The grammar is based on the input and output of pieces of information, it means, from the user side (UI) the user chooses some option or introduces some information from its keyboard, from the server function, the input needs to be received and wrapped inside a function to process it and then send it back to the user side. For that, the UI side has to have a receptor function to show the information.

All the information from the user side contain the keyword InputNameOfTheFunction() or NameOfTheFunctionInput() to catch the information from the server function the input will be catch by functions called Input$tags. The tags from the server side refer to the id of every input function. Also, to send back some information from the server side to the user side, the user side needs to have a function called OutputNameOfFunction() this way from the server the function renders the information and sends it back to the user side. All these systems will be seen in detail later on in the following chapters



FIGURE 8.R SHINY LOGO HTTPS://WWW.RSTUDIO.COM/

Most of the greatest work made in R shiny can be seen at https://shiny.rstudio.com/. The website has become the "Hello world" for beginners. In this work, we will explore some of those solutions to implement them in our model gathering the best of each of them. Two main examples on which this work was based are shown later in this work, to know, Eduard Parker Covid tracker who manages to represent in a timeline and in the same map the evolution of several variables and the ZipCode project from Scholerke who represents every single postcode in a map on which certain variable change depending what location we are at.

LEAFLET

As our goal is to build a map from scratch not using premade tools like tableau or Powerbi. For that, we need to rebuild the wheel in some way and set up every single step on our own. In this case, to build the map we will use a third-party R library called leaflet.

Leaflet is a javascript library created in 2011 by Volodymyr Agafonkin, an Ukrainian citizen living in Kyiv. Leaflet is an open-source library used to build web mapping applications. It supports Web Map Service (WMS) layers, GeoJson layers, Vector layers and tile layers natively.

The majors' leaflet object types are:

- Raster Types (TileLayer and ImageOverlay): It adds an image map of our interest
- Vector Types (Path, Polygon, and specific types such as Circle): It adds the geometry to the map we had set up. The geometries consist of region boundaries on the map.
- Grouped Types (LayerGroup, FeatureGroup, and GeoJson): It allows us to add multiple geometries and features in a single map.
- Controls (Zoom, Layers, etc.): Adds features to control the different layers on the map.

Despite Leaflet being a javascript library is possible to use this library through R using the R's package Leaflet created by Joe Chang (https://github.com/jcheng5) which allow using an interface to easily create leaflet object using R commands.

The leaflet package is an extensive library with its own grammar to manipulate maps and data. The maps are in fact images treated in a smart way. To represent countries or boundaries on a map is necessary to use special data. These data are JSON objects in its majority, every country is a set of polygonal figures represented by an array of vectors that represent the vector trajectory of each line in a map. Other features like longitude and latitude are included in this type of data. In general, such an object is called GeoJSON but this is not the only type of data capable of representing boundaries. The type of object is aligned also with the type of location in the coordinate system. In our case, GeoJSON supports the WGS84 (World Geodesic System) coordinate system but does not support others.

INTRODUCTION



FIGURE 9. DATA ANALYSIS PROCESS HTTPS://WWW.G2.COM/ARTICLES/DATA-ANALYSIS-PROCESS

The Data Analysis process entails the collection, transformation, cleaning, and modelling of data to discover useful and interesting information for an organization. All the data obtained is transformed into conclusions and used for decision-making. To carry out a Data Analysis project there are three main methodologies to follow:

- KDD or Knowledge Discover Database
- SEMMA or Sample, Explore, Modify, Model and Assess
- CRISP-DM

KDD or Knowledge Discover Database It is a process focused on extracting knowledge from a repository, emphasizing high-level applications of certain Data Mining methods.

- Selection: Through this phase, the variables are selected, or if you are going to work with samples, the data that must go to them, their distribution, etc.
- Preprocessing: In this stage, the data is cleaned and adapted to be consumed by the Data Analysis tool.
- Transformation: Here dimensional reduction methods or other transformation methods are used, in order to apply the Data Mining techniques that are chosen.
- Data mining: Here we try to resolve the doubts raised with the chosen data mining techniques, or we start the search for new findings, we try to detect patterns in the data.
- Interpretation/Evaluation: Through this last stage, the conclusions of the study are drawn and the data mining models used are evaluated.

SEMMA o Sample, Explore, Modify, Model and Assess As a methodology it can be applied to any Data Mining process, it is linked to the SAS Software for Data Mining.

It consists of 5 stages:

- Sampling: In this stage, samples of the data are obtained that are representative for the analysis, but of an adequate size to be able to manipulate them in a time and with reasonable resources.
- Explore: Preliminary analysis of the data, obtaining some initial conclusions about its morphology, trends, etc., to help us decide which way to go.
- Modify: In this phase, the data is modified, transformations are applied, and selections are made to create the variables already oriented to the model selection process.
- Model: Applying Data Mining Models, functions or combinations of the variables chosen as predictors are obtained, which helps us to predict the target variable.
- Evaluate: In this last phase, the usefulness and reliability of the insights obtained with the Model are evaluated, and their performance is estimated.

CRISP-DM or Cross Industry Standard Process for Data Mining It is an iterative and business-focused process. It is independent of the Data Mining tool used and is closely tied to SPSS Clementine. It is cyclical and consists of six flexible stages:

- Business Understanding: Understand the objectives and requirements from a Business point of view, as a previous step to the definition of the Data Mining problem.
- Data understanding: Data is collected, and explored, problems with data quality are detected, and the first insights, data subsets, and first hypotheses are obtained.
- Data preparation: A dataset is built from the raw data. Modelling: Modeling techniques are selected and applied, and their parameters are calibrated in search of the best results.
- Evaluation: The models are evaluated, and the steps followed to build the Models are reviewed, in relation to the Business objectives.
- Deployment: The model(s) are “industrialized”. It can be deployed in a production system or environment, or at least a deliverable is generated that the client can use

As we have seen we count on numerous methodologies to implement in our work. The methodologies to follow will be a combination of them taking those elements that better fit our purpose, it means, monitories the trend. The stage to analyse are the following:

1. Defining why you need data analysis:
    a. Defining the goals
2. Begin collecting data from sources
    a. Gathering data
3. Cleaning through unnecessary data
    a. Cleaning data
4. Begin analysing the data:
    a. Creating the plots and monitories the trend
5. Developing the app:
    a. Creating the R shiny template
    b. Developing the visual structure
    c. Deploying the app

Let’s briefly see each of the steps in this section to develop it later on in the following chapters

In the last decade, climate change has become the main concern for many countries. Scientists, activists, groups, and institutions have been sounding the alarm about the changes that lie ahead if countries do not quickly commit to a 180-degree turn in current policies.

Due to this concern, politicians from different countries decided to take a step forward and form an agreement in Paris known as COP21 on December 12, 2015. In said agreement, 196 countries committed themselves to take long-term measures

The measures adopted in the said agreement are defined as follows:

- Substantially reduce greenhouse gas emissions to limit global temperature rise this century to 2°C and strive to limit this increase to even more than just 1.5°.
- review country commitments every five years.
- offer financing to developing countries so that they can reduce climate change, strengthen resilience, and improve their capacity to adapt to the impacts of climate change.

The Agreement is a legally binding international treaty. It entered into force on November 4, 2016. Currently, 193 parties (192 countries plus the European Union) have signed the Paris Agreement.

The agreement in general calls on the different countries to adopt in order to reduce the effects of climate change, and work together to meet the objectives to which they have aligned. The agreement also includes help from developed countries to developing countries to avoid the use of polluting productive technology that is in high demand for developing countries. The agreement also contemplates the use of systems to monitor the development of the objectives.

The Paris Agreement provides a lasting framework to guide the global effort for decades to come. It signals the beginning of a shift to a net-zero emissions world. Implementation of the Agreement is also essential to achieving the Sustainable Development Goals, as it provides a roadmap for climate action that will reduce emissions and change climate resilience.

Under this action framework, the United Kingdom launched a plan called zero CO2 emissions in the transport system by 2040. On1 August 2022, the statement read as follows

A. As governments, we will work towards all sales of new cars and vans being zero emission by 2040 or earlier, or by no later than 2035 in leading markets.

B. As governments in emerging markets and developing economies, we will work intensely towards accelerated proliferation and adoption of zero-emission vehicles. We call on all developed countries to strengthen the collaboration and international support offered to facilitate a global, equitable and just transition.

C. As cities, states, and regional governments, we will work towards converting our owned or leased car and van fleets to zero-emission vehicles by 2035 at the latest, as well as putting in place policies that will enable, accelerate, or otherwise incentivise the transition to zero-emission vehicles as soon as possible, to the extent possible given our jurisdictional powers.

D. As automotive manufacturers, we will work towards reaching 100% zero emission new car and van sales in leading markets by 2035 or earlier, supported by a business strategy that is in line with achieving this ambition, as we help build customer demand.

E. As business fleet owners and operators, or shared mobility platforms, we will work towards 100% of our car and van fleets being zero emission vehicles by 2030, or earlier where markets allow.

F. As investors with significant shareholdings in automotive manufacturers, we will support an accelerated transition to zero-emission vehicles in line with achieving 100% new car and van sales being zero-emission in leading markets by 2035. We will provide proactive engagement and escalation of these issues with investees, coupled with encouraging all our holdings to decarbonise their fleets in line with science-based targets.

G. As financial institutions, we confirm our support for an accelerated transition to zero-emission vehicles in line with achieving 100% new car and van sales being zero-emission in leading markets by 2035, supported by making capital and financial products available to enable this transition for consumers, businesses, charging infrastructure and manufacturers.

H. As other signatories, we support an accelerated transition to zero-emission vehicles in line with achieving 100% of new car and van sales being zero-emission in leading markets by 2035.

our goal is to create a prototype of an app that helps monitor the tracking of objectives under this regulatory framework

Gathering data

To develop the App the first step is to get enough data that allows us to monitor the effectiveness of the measurements taken during the time. In this case, longitudinal data play an essential role. Our main focus will be the demand for an electric vehicle, Electricity supply and demands, CO2 evolution and charge points stock:

- Electricity Data: evolution by the source of type, supply and demand
- CO2 Data: emission by regions
- GPS Data: data location of charge points
- EV Data: Electric vehicle demand.

The following chart resumes their relationship:

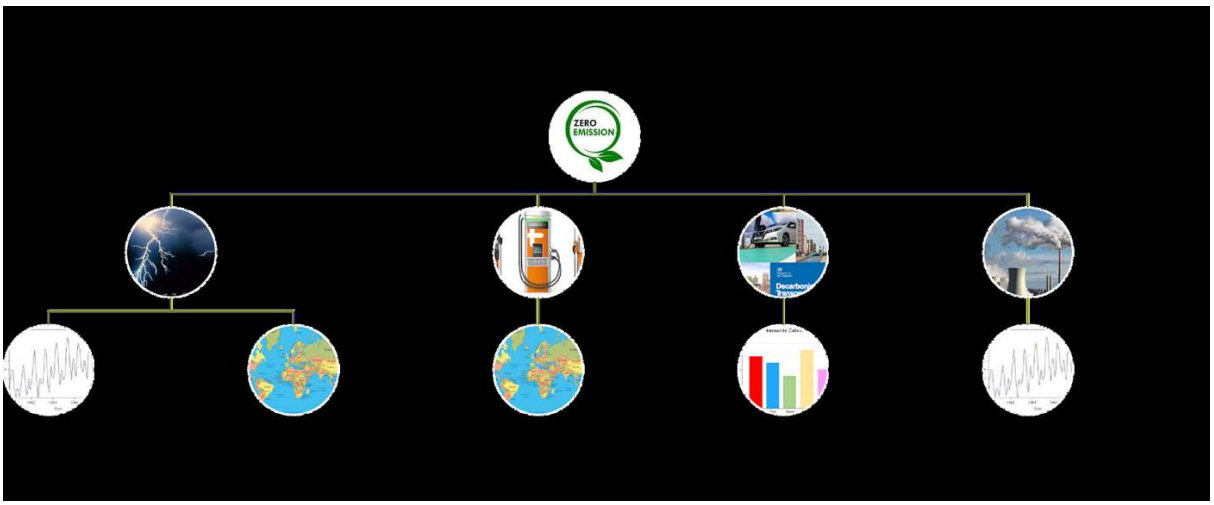


FIGURE 10 DATA RELATIONSHIP (OWN PRODUCTION)

CLEANING DATA

All the data has been acquired from different resources and formats mainly, Excel, JSON, ODT, CSV and others. To manage work with all of them in a single environment is necessary to clean the data, it means leaving it in a meaningful way to exploit them. For that R offers, different packages to achieve this objective like dplyr. Let's see an example by transforming the next table.

Components of electricity generation and supply	2004 UK Total	2004 Scotland	2004 Wales	2004 Northern Ireland	2004 England	2005 UK Total	2005 Scotland	2005 Wales	2005 Northern Ireland	2005 England
Electricity generated by Major Power Producers (MPPs)	358,313	44,453	33,020	7,141	273,699	362,212	41,848	32,254	9,239	278,871
Electricity generated by other generators	35,595	5,484	2,402	272	27,437	36,148	7,390	2,399	381	25,977
Total electricity generated	393,908	49,937	35,422	7,412	301,136	398,360	49,237	34,653	9,620	304,849
Own use by other generators	1,451	224	98	11	1,118	1,608	329	107	17	1,156
Net electricity supplied by other generators	34,144	5,260	2,304	261	26,319	34,539	7,061	2,292	364	24,822
Used in pumping at pumped storage and other own use by MPPs	19,079	3,800	4,177	284	10,818	19,972	3,403	4,606	325	11,638
Net electricity supplied by MPPs	339,235	40,653	28,844	6,857	262,880	342,240	38,445	27,648	8,914	267,233
Electricity transferred to England (net of receipts)	0	5,780	9,777	0	-15,557	0	5,628	8,253	0	-13,881
Electricity transferred to Northern Ireland (net of receipts)	0	2,793	0	-2,793	0	0	1,687	0	-1,687	0
Electricity transferred to Europe (net of receipts)	-7,490	0	0	1,574	-9,064	-8,321	0	0	2,073	-10,394
Transfers from other generators to public supply	10,078	1,391	903	153	7,632	8,537	2,014	981	271	5,270
Transmission losses	5,713	560	321	125	4,707	6,354	630	358	147	5,219
Distribution losses and theft	25,015	1,929	1,146	485	21,455	21,320	1,759	1,094	480	17,986
Consumption from public supply [A]	326,101	30,986	18,505	7,619	268,992	331,455	30,761	18,927	8,173	273,594
Consumption by autogenerators	24,039	3,865	1,400	108	18,666	25,971	5,041	1,309	92	19,529
Total electricity consumed	350,140	34,851	19,904	7,727	287,658	357,426	35,802	20,236	8,265	293,123

Electricity sales (public supply) [B]	*323,714*	30,117	19,067	7,558	266,972	*331,273*	30,975	19,363	7,647	273,288
Statistical difference between calculated consumption [A] and sales [B]	*2,387*	869	-562	61	2,019	*182*	-213	-436	525	306

FIGURE 11 ELECTRICITY SUPPLY BY REGION (OWN PRODUCTION)

Figure 11 shows how year and region are set by columns. It could be interesting to have it in a more meaningful way like the next table for example.

Region	Components	Year	total
England	Total electricity consumed	2016	253678
England	Total electricity consumed	2017	248946
England	Total electricity consumed	2018	248706
England	Total electricity consumed	2019	245293
England	Total electricity consumed	2020	232527
Wales	Total electricity generated	2004	35422
Wales	Total electricity generated	2005	34653
Wales	Total electricity generated	2006	35636
Wales	Total electricity generated	2007	32688

FIGURE 12 ELECTRICITY BY REGION TRANSFORMED (OWN PRODUCTION)

We have exactly the same data but this time we follow the tidy philosophy introduced by Hadley Wickham:

1. Each variable must have its own column: in figure 4 the name column contained an actual value, year, and another value, region, for example, **2004 Scotland**. In this case, we should provide a variable for these two values year, and region and another variable for its observation, total, as in figure 12.
2. Each observation must have its own row: we accomplished this requirement
3. Each value must have its own cell: We accomplished this requirement

The same philosophy is followed by the rest of the dataset.

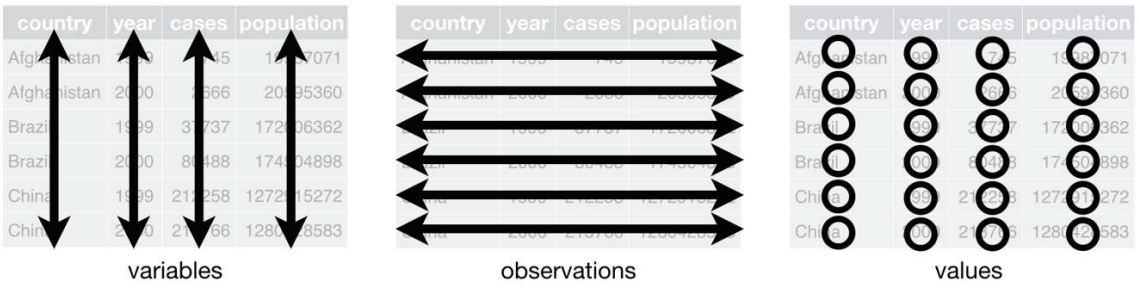


FIGURE 12. R FOR DATA SCIENCE. HADLEY WICKHAM HTTPS://R4DS.HAD.CO.NZ/

Despite R-shiny having been in the market for a decade and continue growing this tool is not still widely used due to its complexity. Such complexity lies in the use of different programming languages to exploit its possibilities though it is thought to use only R code this sometimes does not allow personalizing each product enough using just R.

Languages like Javascript, CSS, and HTML are necessary to make each product pretty different, but such languages normally are dominated by web developers not by data analysts, for that R-shiny templates provide masks that actually wrap those codes to produce the output. For example, to create the next template



Is necessary to write the next script

```
FluidPage(
      TabsetPanel(
       TabPanel(…)
      )
      )
```

The previous code is an R script that wraps HTML and CSS code. For `FluidPage` we have that this sentence wraps the next code `<div class="container-fluid"></div>` which is an HTML tag that creates a division or a section in an HTML document, in similar manner for `TabsetPanel` and `TabPanel` that creates different tabs in the document. This way writing R-code we do not need to use other programming languages, but as I said before, to personalize an App some knowledge in other languages will allow improving the app but this is the main reason why R shiny is not widely used despite its powerful possibilities.

R shiny offer a wide selection of templates which we can use to develop any product from scratch. Many of them can be found at https://shiny.rstudio.com/gallery/ like a part of other projects that have been published by other developers.

In 2013 the WashingtonPost published "*Washington: A world apart*" (https://www.washingtonpost.com/sf/local/2013/11/09/washington-a-world-apart/) in the article an astonishing application was developed to highlight those ZIP codes with the highest ranking on income and college level.

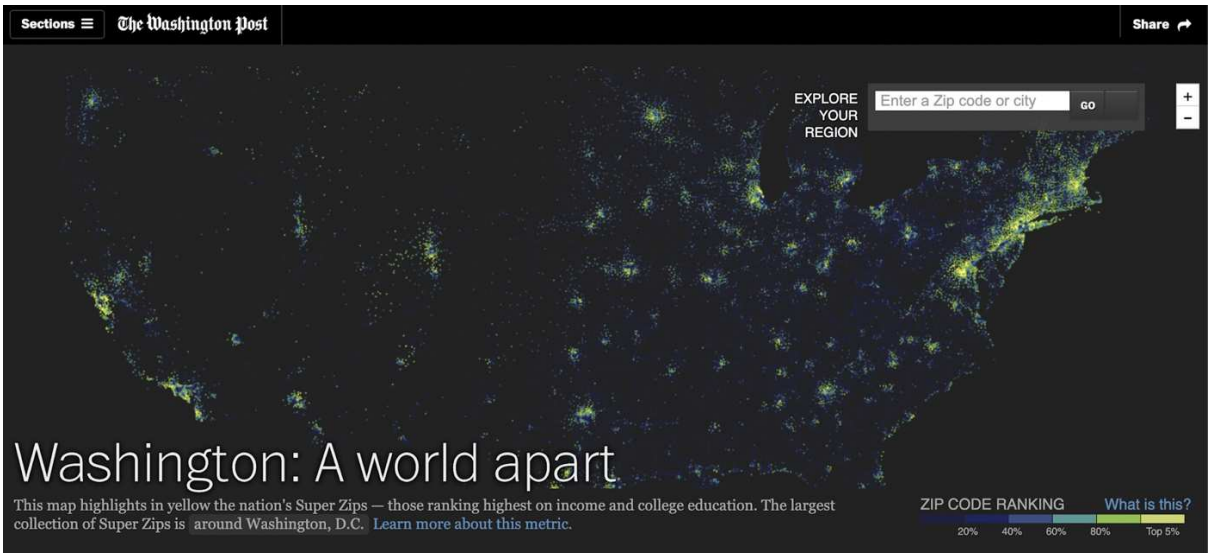


FIGURE 14 WASHINGTON POST ARTICLE HTTPS://WWW.WASHINGTONPOST.COM/SF/LOCAL/2013/11/09/WASHINGTON-A-WORLD-APART

The app was replicated later on by Barret Schloerke (https://github.com/schloerke) using R-shiny but using different files for Javascript, HTML and CSS to expand the possibilities of the application.

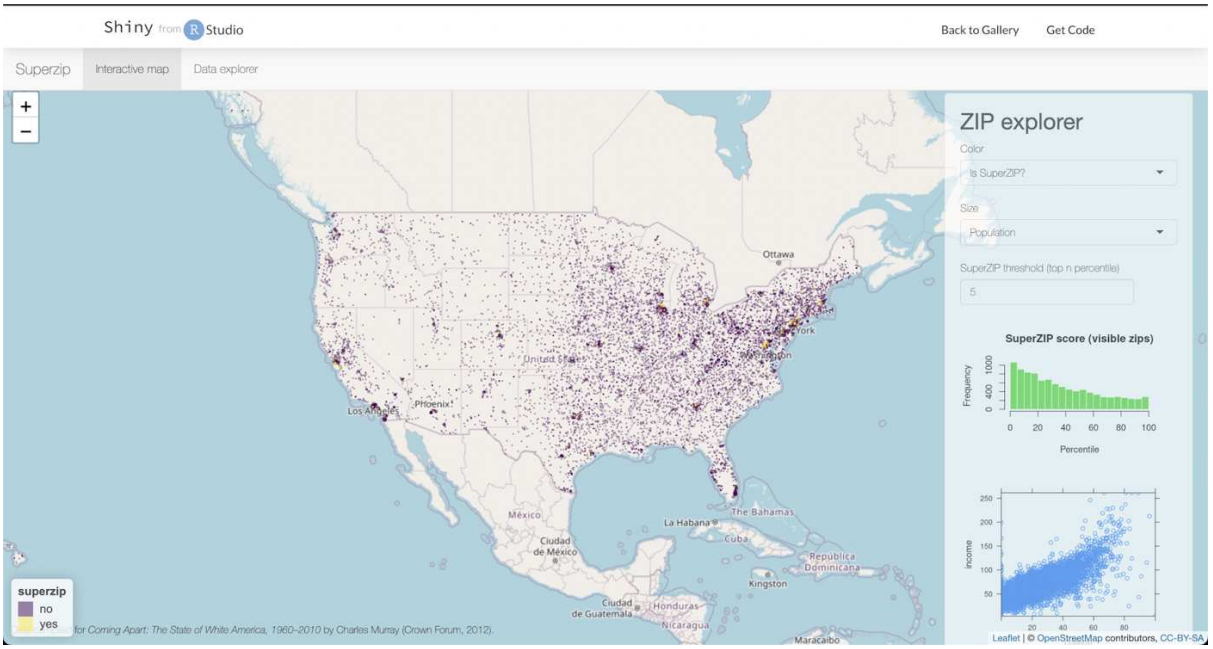


FIGURE 15 BARRET SCHLOERKE : HTTPS://SHINY.RSTUDIO.COM/GALLERY/SUPERZIP-EXAMPLE.HTML

This method to create interactive maps become a kind of standard and many other apps were replicated using this template with other variations and different purposes. In 2020 Edward Parker published *Coronavirus outbreak: a new mapping tool that lets you scroll through a timeline (*[*https://theconversation.com/coronavirus-outbreak-a-new-mapping-tool-that-lets-you-scroll-through-timeline-131422*](https://theconversation.com/coronavirus-outbreak-a-new-mapping-tool-that-lets-you-scroll-through-timeline-131422)*)* In this article, Parker add a time dimension to the analysis allowing see the evolution of the variables in the map and therefore track visually and spatially the interactions.



FIGURE 16. EDWARD PARKERHTTPS://SHINY.RSTUDIO.COM/GALLERY/COVID19-TRACKER.HTML

In our work, we will use a combination of these perspectives to track the UK 2030 Zero emission objective.

Deploying the model

The last stage of the project is to deploy the results on the web. We can choose our own server, or we can host it on their hosting service for free

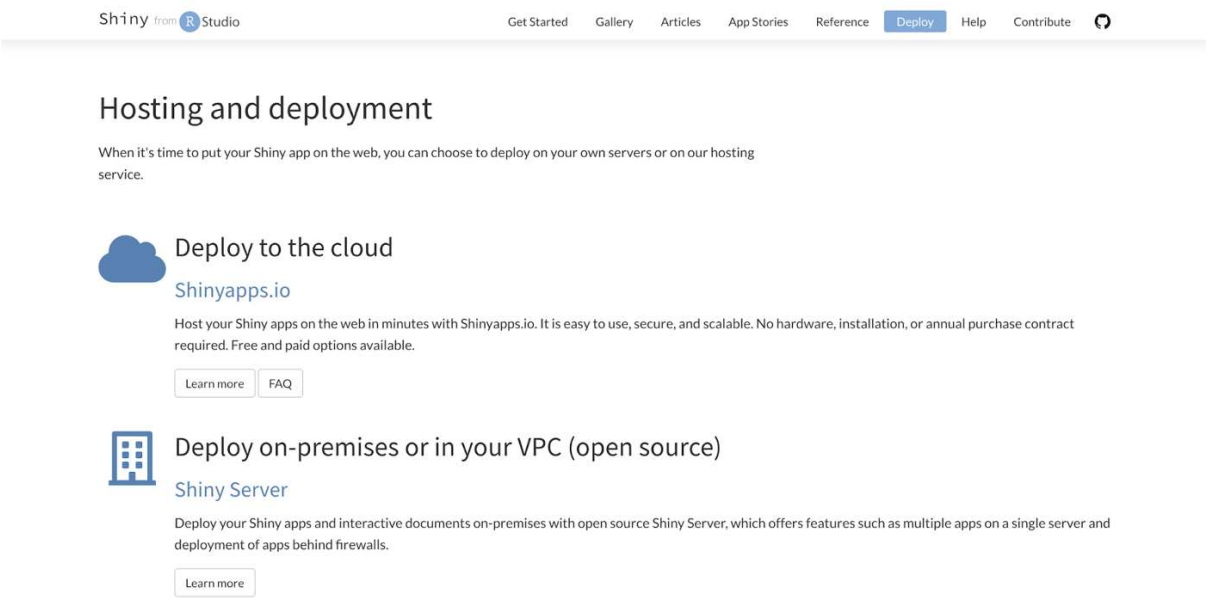


Figure 2 Rstudio server shinyio.rstudio.com

Once the project is deployed it can be accessed by thirds parties

Chapter 4: Implementation

Introduction

Let's start implementing our App following the steps shown in chapter 3.

Gathering Data

Countries boundaries

Since our objective will be to build a map to represent our features the first step will be to build our map framework. For that, we will need to define at what region level we want to work. In this case, we will work for countries levels in the United Kingdom: England, Scotland, Wales, and Northern Ireland.

As said before, our map will be an Image, which means a Raster Type object in the JavaScript library Leaflet, to work on that image we need to define the country's boundaries. This can be done using the service provided by https://borders.ukdataservice.ac.uk/bds.html which allows downloading files to draw boundaries in a map.

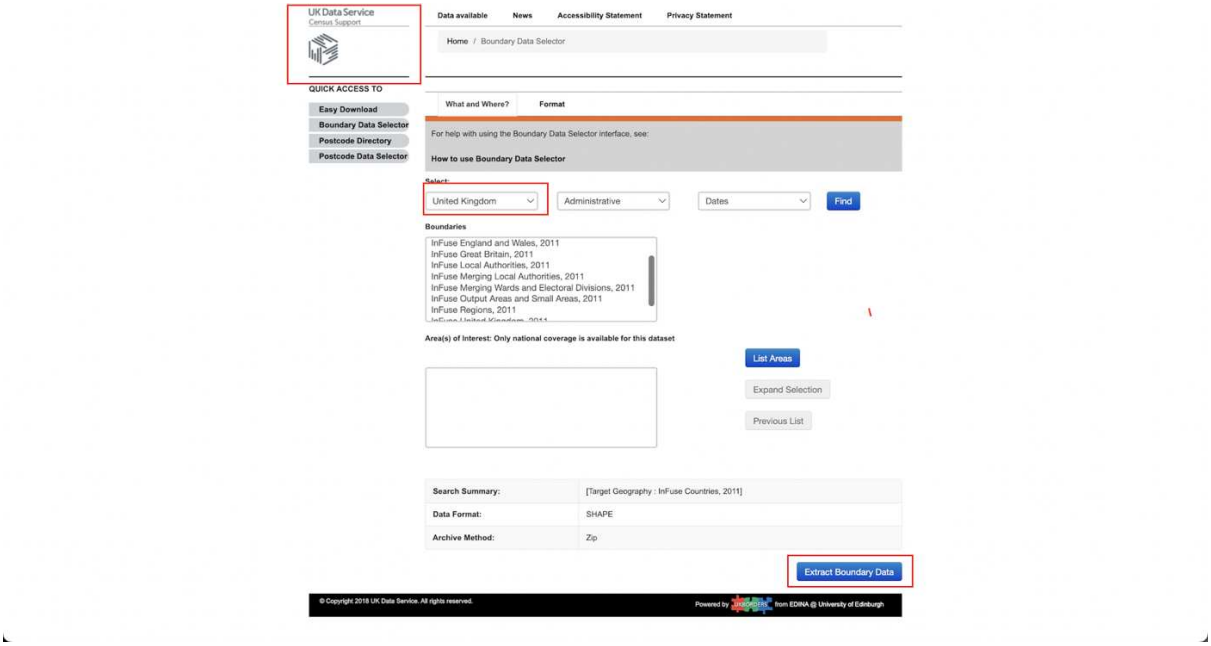


Figure 18 https://borders.ukdataservice.ac.uk/bds.html

Once we select our requirements, we download the file in our case "infuse_countries_2001.zip". The files consist of different files that make it possible to draw boundaries on the image.

- **Infuse_ctry_2011.dbf:** Is a kind of look-up table that maps geocode into countries



FIGURE 19 INFUSE_CTRY_2011.DBF

- Infuse_ctry_2011.prj: This file contains the instruction for the reader software to tell them how the data are organized. It means once it is read from R this file automatically tells the software how to proceed to read all the files.
- Infuse_ctry_2011.shp/shx: Object created and/or used by various 3D design programs. In this case, is used to draw the boundaries.

ELECTRICITY CONSUMPTION BY REGION

Our next data set is “subnational_electricity_consumption_statistics_2005-2020.xlsx” this dataset can be acquired at https://www.gov.uk/government/statistical-data-sets/regional-and-local-authority-electricity-consumption-statistics. The file is an excel file with different sheets for the different years which make it a challenge to organize the data and leave it ‘Tidy’.

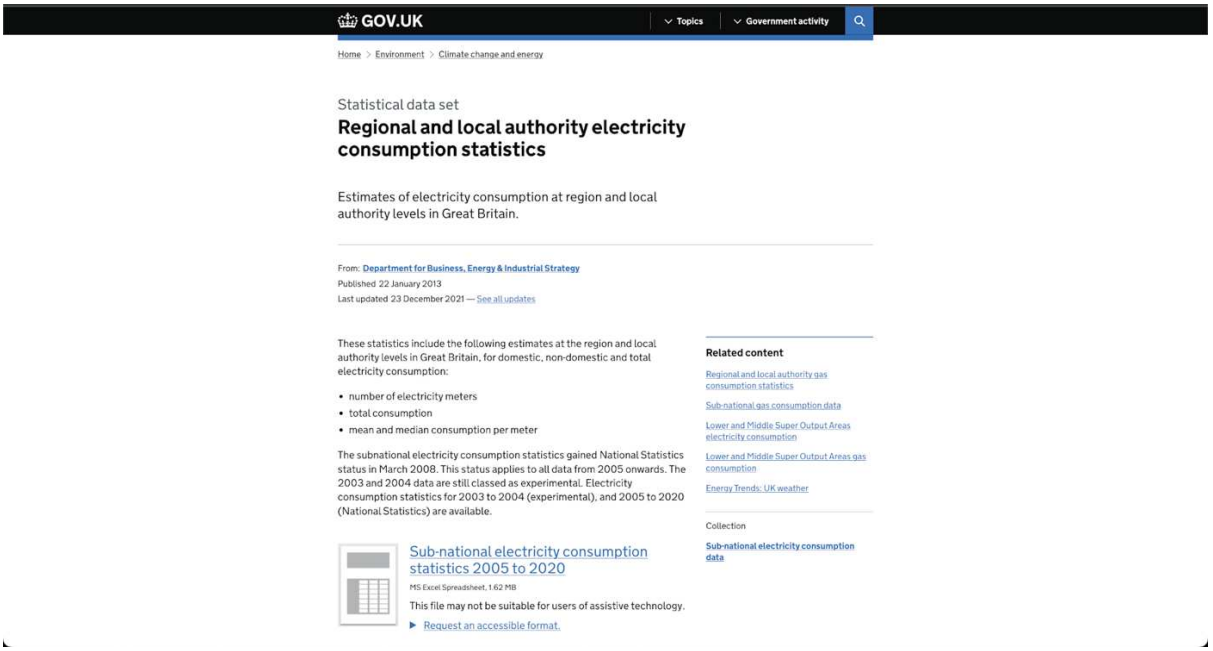
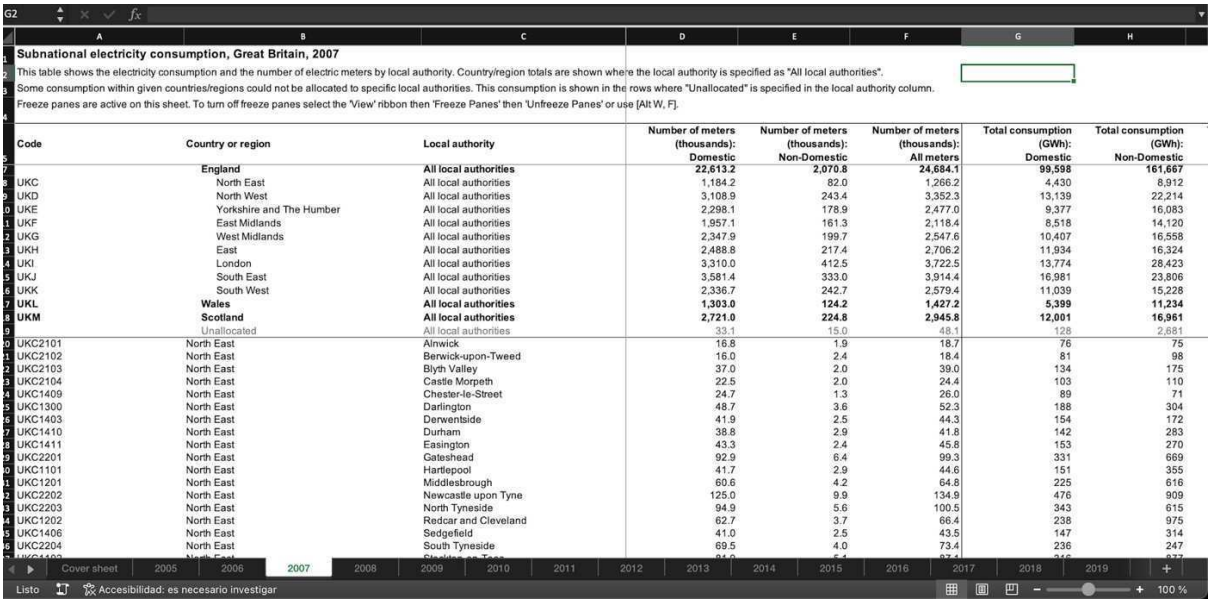


FIGURE 20 HTTPS://WWW.GOV.UK/GOVERNMENT/STATISTICAL-DATA-SETS/REGIONAL-AND-LOCAL-AUTHORITY-ELECTRICITY-CONSUMPTION-STATISTICS



Subnational electricity consumption, Great Britain, 2007

This table shows the electricity consumption and the number of electric meters by local authority. Country/region totals are shown where the local authority is specified as "All local authorities".
Some consumption within given countries/regions could not be allocated to specific local authorities. This consumption is shown in the rows where "Unallocated" is specified in the local authority column.
Freeze panes are active on this sheet. To turn off freeze panes select the 'View' ribbon then 'Freeze Panes' then 'Unfreeze Panes' or use [Alt W, F].

Code	Country or region	Local authority	Number of meters (thousands): Domestic	Number of meters (thousands): Non-Domestic	Number of meters (thousands): All meters	Total consumption (GWh): Domestic	Total consumption (GWh): Non-Domestic
	England	**All local authorities**	**22,613.2**	**2,070.8**	**24,684.1**	**99,598**	**161,667**
UKC	North East	All local authorities	1,184.2	82.0	1,266.2	4,430	8,912
UKD	North West	All local authorities	3,108.9	243.4	3,352.3	13,139	22,214
UKE	Yorkshire and The Humber	All local authorities	2,298.1	178.9	2,477.0	9,377	16,083
UKF	East Midlands	All local authorities	1,957.1	161.3	2,118.4	8,518	14,120
UKG	West Midlands	All local authorities	2,347.9	199.7	2,547.6	10,407	16,558
UKH	East	All local authorities	2,488.8	217.4	2,706.2	11,934	16,324
UKI	London	All local authorities	3,310.0	412.5	3,722.5	13,774	28,423
UKJ	South East	All local authorities	3,581.4	333.0	3,914.4	16,981	23,806
UKK	South West	All local authorities	2,336.7	242.7	2,579.4	11,039	15,228
UKL	**Wales**	**All local authorities**	**1,303.0**	**124.2**	**1,427.2**	**5,399**	**11,234**
UKM	**Scotland**	**All local authorities**	**2,721.0**	**224.8**	**2,945.8**	**12,001**	**16,961**
	Unallocated	All local authorities	33.1	15.0	48.1	128	2,681
UKC2101	North East	Alnwick	16.8	1.9	18.7	76	75
UKC2102	North East	Berwick-upon-Tweed	16.0	2.4	18.4	81	98
UKC2103	North East	Blyth Valley	37.0	2.0	39.0	134	175
UKC2104	North East	Castle Morpeth	22.5	2.0	24.4	103	110
UKC1409	North East	Chester-le-Street	24.7	1.3	26.0	89	71
UKC1300	North East	Darlington	48.7	3.6	52.3	188	304
UKC1403	North East	Derwentside	41.9	2.5	44.3	154	172
UKC1410	North East	Durham	38.8	2.9	41.8	142	283
UKC1411	North East	Easington	43.3	2.4	45.8	153	270
UKC2201	North East	Gateshead	92.9	6.4	99.3	331	669
UKC1101	North East	Hartlepool	41.7	2.9	44.6	151	355
UKC1201	North East	Middlesbrough	60.6	4.2	64.8	225	616
UKC2202	North East	Newcastle upon Tyne	125.0	9.9	134.9	476	909
UKC2203	North East	North Tyneside	94.9	5.6	100.5	343	615
UKC1202	North East	Redcar and Cleveland	62.7	3.7	66.4	238	975
UKC1406	North East	Sedgefield	41.0	2.5	43.5	147	314
UKC2204	North East	South Tyneside	69.5	4.0	73.4	236	247

FIGURE 21 SUBNATIONAL_ELECTRICITY_CONSUMPTION_STATISTICS_2005-2020.XLSX (OWN PRODUCTION)

CO2 EMISSIONS

The CO2 emission data can be found at https://www.data.gov.uk/dataset/723c243d-2f1a-4d27-8b61-cdb93e5b10ff/uk-local-authority-and-regional-greenhouse-gas-emissions the file is a CSV file that gathers greenhouse emission for the different regions and different sectors. The relevant variables for our purpose will be:

- Country
- Calendar Year
- Sector (Transport)

- Green house gas (CO2)

	A	B	C	D	E	F	G	H	I	J
	uk-local-authority-ghg-emissions-									
1	**Country**	**Country Code**	**Region**	**Region Code**	**Second Tier Authority**	**Local Authority**	**Local Authority Code**	**Calendar Year**	**LA GHG Sector**	**LA GHG Sub-sector**
2	**England**	E92000001	East Midlands	E12000004	Derby	Derby	E06000015	2005	Agriculture	Agriculture Electricity
3	**England**	E92000001	East Midlands	E12000004	Derby	Derby	E06000015	2005	Agriculture	Agriculture Electricity
4	**England**	E92000001	East Midlands	E12000004	Derby	Derby	E06000015	2005	Agriculture	Agriculture Electricity
5	**England**	E92000001	East Midlands	E12000004	Derby	Derby	E06000015	2005	Agriculture	Agriculture Gas
6	**England**	E92000001	East Midlands	E12000004	Derby	Derby	E06000015	2005	Agriculture	Agriculture Gas
7	**England**	E92000001	East Midlands	E12000004	Derby	Derby	E06000015	2005	Agriculture	Agriculture Gas
8	**England**	E92000001	East Midlands	E12000004	Derby	Derby	E06000015	2005	Agriculture	Agriculture 'Other'
9	**England**	E92000001	East Midlands	E12000004	Derby	Derby	E06000015	2005	Agriculture	Agriculture 'Other'
10	**England**	E92000001	East Midlands	E12000004	Derby	Derby	E06000015	2005	Agriculture	Agriculture 'Other'
11	**England**	E92000001	East Midlands	E12000004	Derby	Derby	E06000015	2005	Agriculture	Agriculture Soils
12	**England**	E92000001	East Midlands	E12000004	Derby	Derby	E06000015	2005	Commercial	Commercial Electricity
13	**England**	E92000001	East Midlands	E12000004	Derby	Derby	E06000015	2005	Commercial	Commercial Electricity
14	**England**	E92000001	East Midlands	E12000004	Derby	Derby	E06000015	2005	Commercial	Commercial Electricity
15	**England**	E92000001	East Midlands	E12000004	Derby	Derby	E06000015	2005	Commercial	Commercial Gas
16	**England**	E92000001	East Midlands	E12000004	Derby	Derby	E06000015	2005	Commercial	Commercial Gas
17	**England**	E92000001	East Midlands	E12000004	Derby	Derby	E06000015	2005	Commercial	Commercial Gas
18	**England**	E92000001	East Midlands	E12000004	Derby	Derby	E06000015	2005	Commercial	Commercial 'Other'
19	**England**	E92000001	East Midlands	E12000004	Derby	Derby	E06000015	2005	Commercial	Commercial 'Other'
20	**England**	E92000001	East Midlands	E12000004	Derby	Derby	E06000015	2005	Commercial	Commercial 'Other'
21	**England**	E92000001	East Midlands	E12000004	Derby	Derby	E06000015	2005	Domestic	Domestic Electricity
22	**England**	E92000001	East Midlands	E12000004	Derby	Derby	E06000015	2005	Domestic	Domestic Electricity
23	**England**	E92000001	East Midlands	E12000004	Derby	Derby	E06000015	2005	Domestic	Domestic Electricity
24	**England**	E92000001	East Midlands	E12000004	Derby	Derby	E06000015	2005	Domestic	Domestic Gas
25	**England**	E92000001	East Midlands	E12000004	Derby	Derby	E06000015	2005	Domestic	Domestic Gas
26	**England**	E92000001	East Midlands	E12000004	Derby	Derby	E06000015	2005	Domestic	Domestic Gas
27	**England**	E92000001	East Midlands	E12000004	Derby	Derby	E06000015	2005	Domestic	Domestic 'Other'
28	**England**	E92000001	East Midlands	E12000004	Derby	Derby	E06000015	2005	Domestic	Domestic 'Other'
29	**England**	E92000001	East Midlands	E12000004	Derby	Derby	E06000015	2005	Domestic	Domestic 'Other'
30	**England**	E92000001	East Midlands	E12000004	Derby	Derby	E06000015	2005	Industry	Industry Electricity

FIGURE 22 GREENHOUSE GAS EMISSION DATA

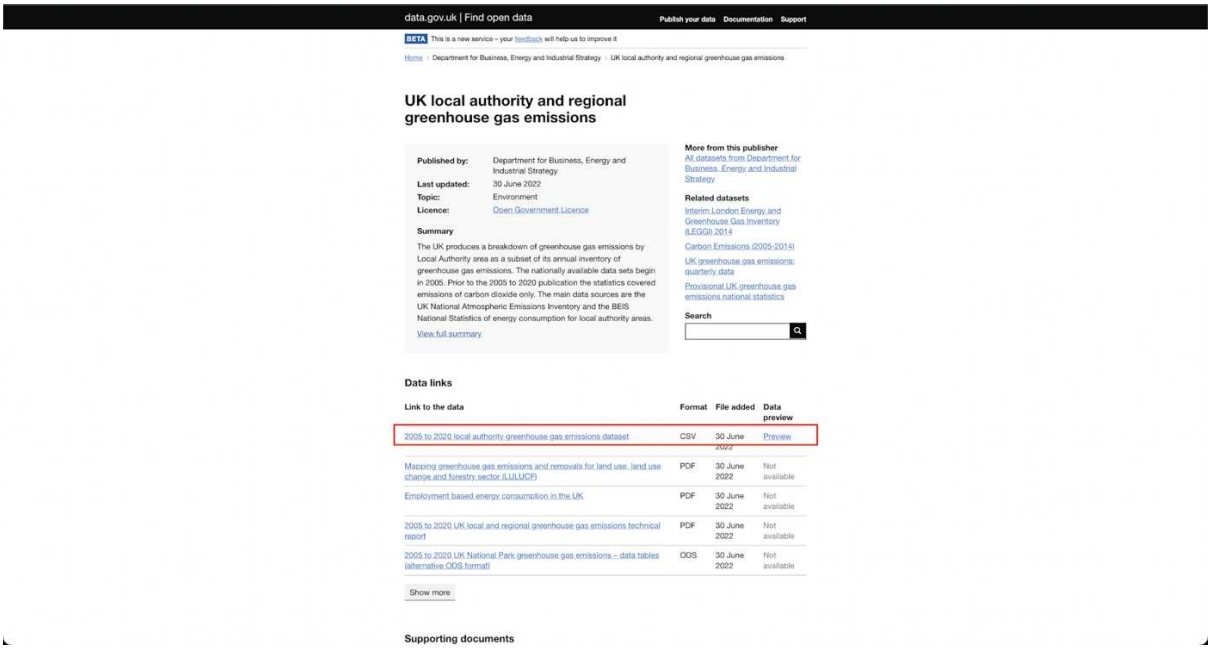


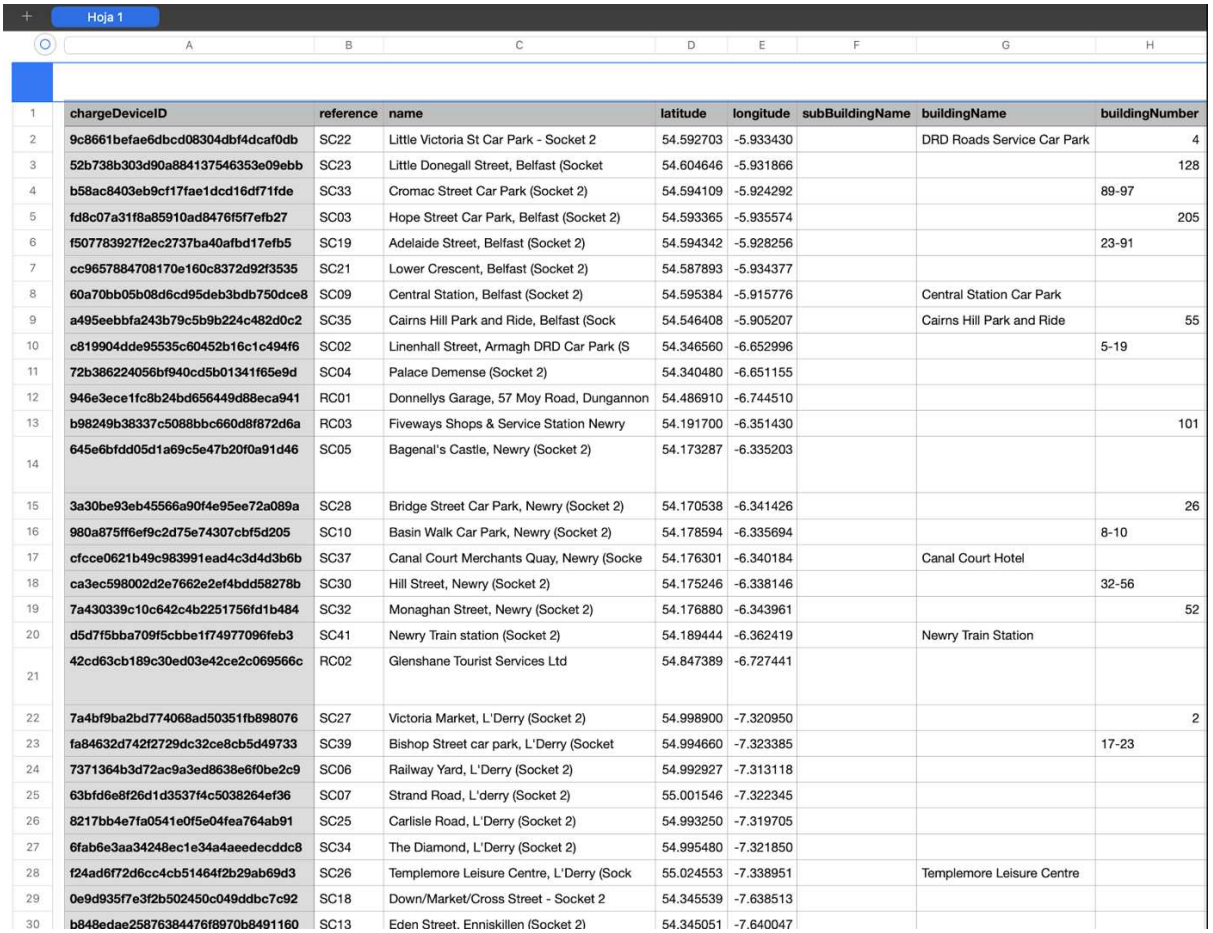
FIGURE 23 HTTPS://WWW.DATA.GOV.UK/DATASET/723C243D-2F1A-4D27-8B61-CDB93E5B10FF/UK-LOCAL-AUTHORITY-AND-REGIONAL-GREENHOUSE-GAS-EMISSIONS

CHARGE POINT GPS

The charge point GPS consists of all the information related to every single charge point in the UK the data can be downloaded at https://datacac.uk/csvTable/united-kingdom/The-entire-dataset-as-CSV62c3048d-4dac-4c0f-aea0-36c4b5e4e73b-39.html. The most relevant variables are:

- Name: street
- Latitude
- Longitude
- DateCreated
- DeviceOwner

This dataset will allow us to map the charge points in the image.

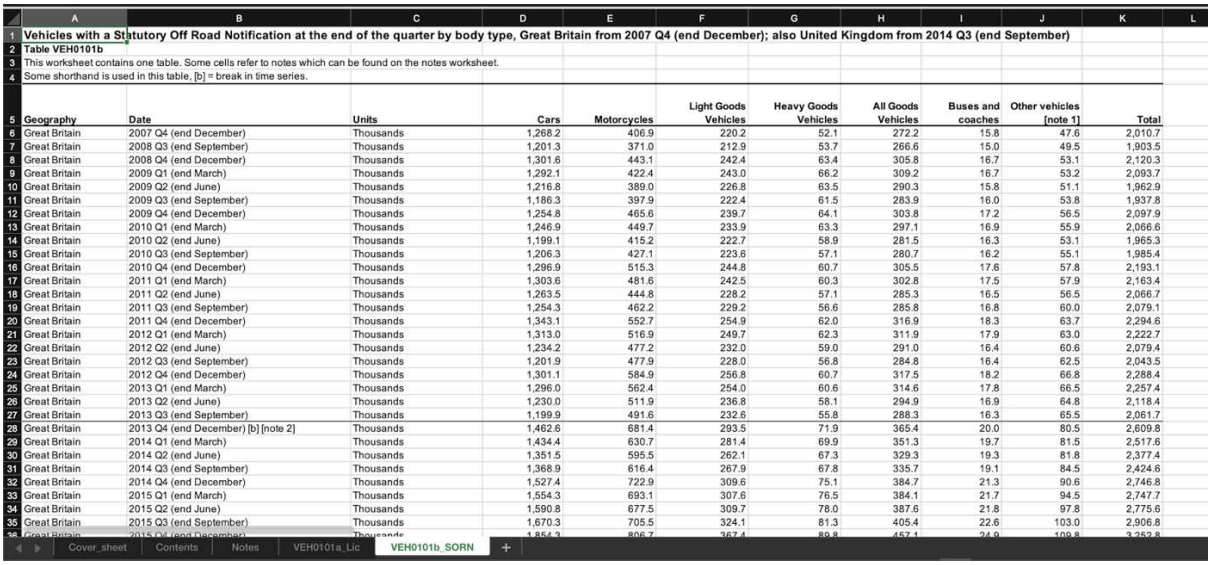


	chargeDeviceID	reference	name	latitude	longitude	subBuildingName	buildingName	buildingNumber
2	9c8661befae6dbcd08304dbf4dcaf0db	SC22	Little Victoria St Car Park - Socket 2	54.592703	-5.933430		DRD Roads Service Car Park	4
3	52b738b303d90a884137546353e09ebb	SC23	Little Donegall Street, Belfast (Socket	54.604646	-5.931866			128
4	b58ac8403eb9cf17fae1dcd16df71fde	SC33	Cromac Street Car Park (Socket 2)	54.594109	-5.924292			89-97
5	fd8c07a31f8a85910ad8476f5f7efb27	SC03	Hope Street Car Park, Belfast (Socket 2)	54.593365	-5.935574			205
6	f507783927f2ec2737ba40afbd17efb5	SC19	Adelaide Street, Belfast (Socket 2)	54.594342	-5.928256			23-91
7	cc9657884708170e160c8372d92f3535	SC21	Lower Crescent, Belfast (Socket 2)	54.587893	-5.934377			
8	60a70bb05b08d6cd95deb3bdb750dce8	SC09	Central Station, Belfast (Socket 2)	54.595384	-5.915776		Central Station Car Park	
9	a495eebbfa243b79c5b9b224c482d0c2	SC35	Cairns Hill Park and Ride, Belfast (Sock	54.546408	-5.905207		Cairns Hill Park and Ride	55
10	c819904dde95535c60452b16c1c494f6	SC02	Linenhall Street, Armagh DRD Car Park (S	54.346560	-6.652996			5-19
11	72b386224056bf940cd5b01341f65e9d	SC04	Palace Demense (Socket 2)	54.340480	-6.651155			
12	946e3ece1fc8b24bd656449d88eca941	RC01	Donnellys Garage, 57 Moy Road, Dungannon	54.486910	-6.744510			
13	b98249b38337c5088bbc660d8f872d6a	RC03	Fiveways Shops & Service Station Newry	54.191700	-6.351430			101
14	645e6bfdd05d1a69c5e47b20f0a91d46	SC05	Bagenal's Castle, Newry (Socket 2)	54.173287	-6.335203			
15	3a30be93eb45566a90f4e95ee72a089a	SC28	Bridge Street Car Park, Newry (Socket 2)	54.170538	-6.341426			26
16	980a875ff6ef9c2d75e74307cbf5d205	SC10	Basin Walk Car Park, Newry (Socket 2)	54.178594	-6.335694			8-10
17	cfcce0621b49c983991ead4c3d4d3b6b	SC37	Canal Court Merchants Quay, Newry (Socke	54.176301	-6.340184		Canal Court Hotel	
18	ca3ec598002d2e7662e2ef4bdd58278b	SC30	Hill Street, Newry (Socket 2)	54.175246	-6.338146			32-56
19	7a430339c10c642c4b2251756fd1b484	SC32	Monaghan Street, Newry (Socket 2)	54.176880	-6.343961			52
20	d5d7f5bba709f5cbbe1f74977096feb3	SC41	Newry Train station (Socket 2)	54.189444	-6.362419		Newry Train Station	
21	42cd63cb189c30ed03e42ce2c069566c	RC02	Glenshane Tourist Services Ltd	54.847389	-6.727441			
22	7a4bf9ba2bd774068ad50351fb898076	SC27	Victoria Market, L'Derry (Socket 2)	54.998900	-7.320950			2
23	fa84632d742f2729dc32ce8cb5d49733	SC39	Bishop Street car park, L'Derry (Socket	54.994660	-7.323385			17-23
24	7371364b3d72ac9a3ed8638e6f0be2c9	SC06	Railway Yard, L'Derry (Socket 2)	54.992927	-7.313118			
25	63bfd6e8f26d1d3537f4c5038264ef36	SC07	Strand Road, L'derry (Socket 2)	55.001546	-7.322345			
26	8217bb4e7fa0541e0f5e04fea764ab91	SC25	Carlisle Road, L'Derry (Socket 2)	54.993250	-7.319705			
27	6fab6e3aa34248ec1e34a4aeedecddc8	SC34	The Diamond, L'Derry (Socket 2)	54.995480	-7.321850			
28	f24ad6f72d6cc4cb51464f2b29ab69d3	SC26	Templemore Leisure Centre, L'Derry (Sock	55.024553	-7.338951		Templemore Leisure Centre	
29	0e9d935f7e3f2b502450c049ddbc7c92	SC18	Down/Market/Cross Street - Socket 2	54.345539	-7.638513			
30	b848edae25876384476f8970b8491160	SC13	Eden Street, Enniskillen (Socket 2)	54.345051	-7.640047			

FIGURE 24 CHARGE POINTS (OWN PRODUCTION)

ELECTRIC VEHICLES

The electric vehicle dataset can be obtained at https://www.gov.uk/government/statistical-data-sets/vehicle-licensing-statistics-data-tables. The dataset records the total electric vehicle by type and year or quarter.



Vehicles with a Statutory Off Road Notification at the end of the quarter by body type, Great Britain from 2007 Q4 (end December); also United Kingdom from 2014 Q3 (end September)										
Table VEH0101b										
This worksheet contains one table. Some cells refer to notes which can be found on the notes worksheet.										
Some shorthand is used in this table, [b] = break in time series.										
Geography	Date	Units	Cars	Motorcycles	Light Goods Vehicles	Heavy Goods Vehicles	All Goods Vehicles	Buses and coaches	Other vehicles [note 1]	Total
Great Britain	2007 Q4 (end December)	Thousands	1,268.2	406.9	220.2	52.1	272.2	15.8	47.6	2,010.7
Great Britain	2008 Q3 (end September)	Thousands	1,201.3	371.0	212.9	53.7	266.6	15.0	49.5	1,903.5
Great Britain	2008 Q4 (end December)	Thousands	1,301.6	443.1	242.4	63.4	305.8	16.7	53.1	2,120.3
Great Britain	2009 Q1 (end March)	Thousands	1,292.1	422.4	243.0	66.2	309.2	16.7	53.2	2,093.7
Great Britain	2009 Q2 (end June)	Thousands	1,216.8	389.0	226.8	63.5	290.3	15.8	51.1	1,962.9
Great Britain	2009 Q3 (end September)	Thousands	1,186.3	397.9	222.4	61.5	283.9	16.0	53.8	1,937.8
Great Britain	2009 Q4 (end December)	Thousands	1,254.8	465.6	239.7	64.1	303.8	17.2	56.5	2,097.9
Great Britain	2010 Q1 (end March)	Thousands	1,246.9	449.7	233.9	63.3	297.1	16.9	55.9	2,066.6
Great Britain	2010 Q2 (end June)	Thousands	1,199.1	415.2	222.7	58.9	281.5	16.3	53.1	1,965.3
Great Britain	2010 Q3 (end September)	Thousands	1,206.3	427.1	223.6	57.1	280.7	16.2	55.1	1,985.4
Great Britain	2010 Q4 (end December)	Thousands	1,296.9	515.3	244.8	60.7	305.5	17.6	57.8	2,193.1
Great Britain	2011 Q1 (end March)	Thousands	1,303.6	481.6	242.5	60.3	302.8	17.5	57.9	2,163.4
Great Britain	2011 Q2 (end June)	Thousands	1,263.5	444.8	228.2	57.1	285.3	16.5	56.5	2,066.7
Great Britain	2011 Q3 (end September)	Thousands	1,254.3	462.2	229.2	56.6	285.8	16.8	60.0	2,079.1
Great Britain	2011 Q4 (end December)	Thousands	1,343.1	552.7	254.9	62.0	316.9	18.3	63.7	2,294.6
Great Britain	2012 Q1 (end March)	Thousands	1,313.0	516.9	249.7	62.3	311.9	17.9	63.0	2,222.7
Great Britain	2012 Q2 (end June)	Thousands	1,234.2	477.2	232.0	59.0	291.0	16.4	60.6	2,079.4
Great Britain	2012 Q3 (end September)	Thousands	1,201.9	477.9	228.0	56.8	284.8	16.4	62.5	2,043.5
Great Britain	2012 Q4 (end December)	Thousands	1,301.1	584.9	256.8	60.7	317.5	18.2	66.8	2,288.4
Great Britain	2013 Q1 (end March)	Thousands	1,296.0	562.4	254.0	60.6	314.6	17.8	66.5	2,257.4
Great Britain	2013 Q2 (end June)	Thousands	1,230.0	511.9	236.8	58.1	294.9	16.9	64.8	2,118.4
Great Britain	2013 Q3 (end September)	Thousands	1,199.9	491.6	232.6	55.8	288.3	16.3	65.5	2,061.7
Great Britain	2013 Q4 (end December) [b] [note 2]	Thousands	1,462.6	681.4	293.5	71.9	365.4	20.0	80.5	2,609.8
Great Britain	2014 Q1 (end March)	Thousands	1,434.4	630.7	281.4	69.9	351.3	19.7	81.5	2,517.6
Great Britain	2014 Q2 (end June)	Thousands	1,351.5	595.5	262.1	67.3	329.3	19.3	81.8	2,377.4
Great Britain	2014 Q3 (end September)	Thousands	1,368.9	616.4	267.9	67.8	335.7	19.1	84.5	2,424.6
Great Britain	2014 Q4 (end December)	Thousands	1,527.4	722.9	309.6	75.1	384.7	21.3	90.6	2,746.8
Great Britain	2015 Q1 (end March)	Thousands	1,554.3	693.1	307.6	76.5	384.1	21.7	94.5	2,747.7
Great Britain	2015 Q2 (end June)	Thousands	1,590.8	677.5	309.7	78.0	387.6	21.8	97.8	2,775.6
Great Britain	2015 Q3 (end September)	Thousands	1,670.3	705.5	324.1	81.3	405.4	22.6	103.0	2,906.8

Cover_sheet | Contents | Notes | VEH0101a_Lic | VEH0101b_SORN | +

FIGURE 25 ELECTRIC VEHICLE DATASET (OWN PRODUCTION)

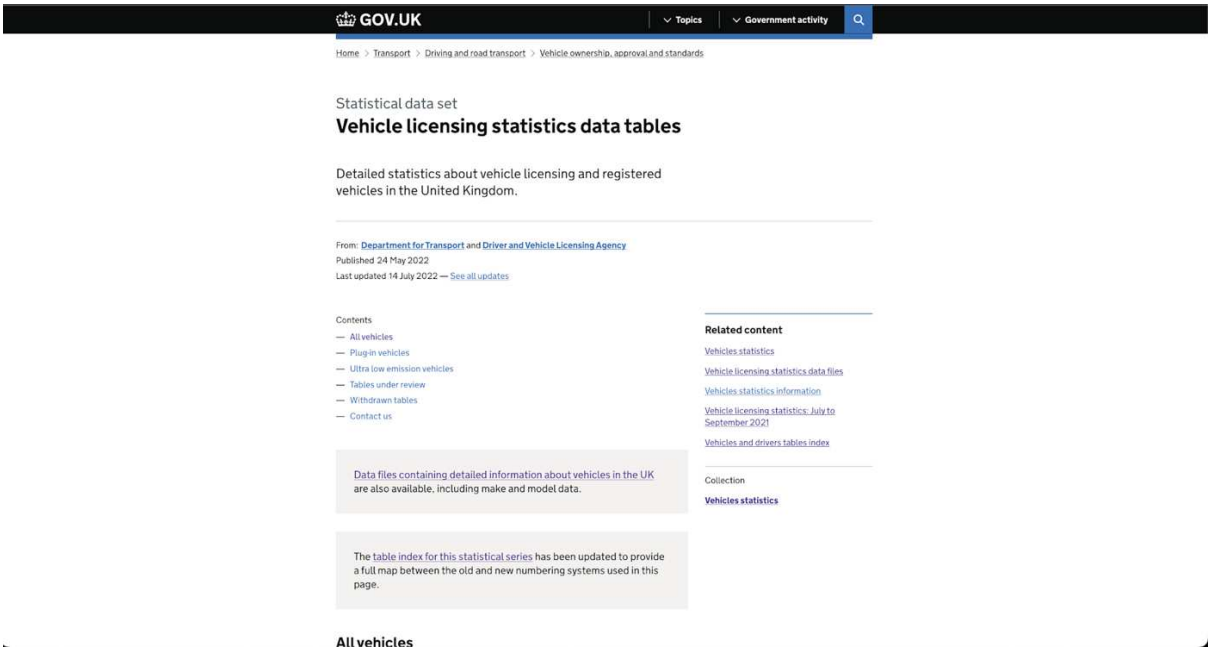


FIGURE 26 AT HTTPS://WWW.GOV.UK/GOVERNMENT/STATISTICAL-DATA-SETS/VEHICLE-LICENSING-STATISTICS-DATA-TABLES

CLEANING DATA

INTRODUCTION

In the previous section, we gathered all the data needed for the App. In this section, we will proceed to clean the data a leave them in a meaningful way to work with them.

COUNTRIES BOUNDARIES

Before we saw how we download the file “infuse_countries_2001.zip” this file by itself is not enough to work with it. It will be necessary to transform it into another object to handle all the information. In 2017, Kan Nishida face the problem to find a JSON object that draws properly the countries boundary of the united kingdom and he post his solution at https://blog.exploratory.io/making-maps-for-uk-countries-and-local-authorities-areas-in-r-b7d222939597. In this article he precisely describes every step to follow to get the object. Let’s follow the step

In the first place, our “infuse_countries_2001.zip” files come in the easting/northing system of the British National Grid, which is based on OSGB 36 datum (1936 Ordnance Survey Great Britain), which is not the one Geojson supports and we need it this format to work with the map

```
#libraries
if(!require(shinyWidgets)) install.packages("shinyWidgets", repos = "http://cran.us.r-project.org")

library( 'spdplyr')
library(rgdal)
library(tidyverse)
library(leaflet)
library(sf)
library(readxl)
library(readxl)
library(lubridate)
library(rgdal)
if(!require(geojsonio)) install.packages("geojsonio", repos = "http://cran.us.r-project.org")
#mark from towards science
https://blog.exploratory.io/making-maps-for-uk-countries-and-local-authorities-areas-in-r-b7d222939597
uk_country <- readOGR(dsn = "/Users/eduardromero/Library/Mobile Documents/com~apple~CloudDocs/Documents/London
Metropolitan University/Msc_production_project/data_processing/data_processing/data/infuse_ctry_2011_1",
                      layer = "infuse_ctry_2011")
```

Once we load the file we swap from the ‘OSGB36’ coordinate system to Longitude / Latitude points coordinate system based on ‘WGS84’ (World Geodetic System 1984) datum because GeoJSON supports only this coordinate system. Let’s transform it into ‘WGS84’

```
# Convert to Longitude / Latitude with WGS84 Coordinate System
wgs84 = '+proj=longlat +datum=WGS84'
uk_country_trans <- spTransform(uk_country, CRS(wgs84))
```

Now we can transform it into a GeoJSON object to work with maps and after that into a Tibble object to make it in a table format

```
# Convert from Spatial Dataframe to GeoJSON

uk_country_json <- geojson_json(uk_country_trans)
uk_json <- uk_country_json %>%st_as_sf()%>%tibble()
```

Let’s see the result

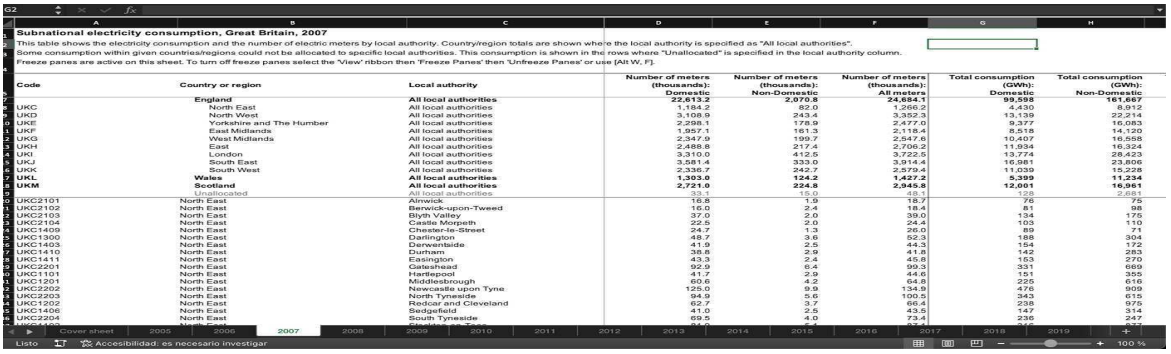
```
> uk_json
# A tibble: 4 × 7
  geo_code  geo_label        geo_labelw       label     name             rmapshaperid                                  geometry
  <chr>     <chr>            <chr>            <chr>     <chr>                   <int>                       <MULTIPOLYGON [°]>
1 N92000002 Northern Ireland Gogledd Iwerddon N92000002 Northern Ireland            0 (((-5.4412 54.4773, -5.442 54.4756, -5.442 54.4…
2 S92000003 Scotland         Yr Alban         S92000003 Scotland                    1 (((-2.0332 55.8117, -2.0344 55.8111, -2.0345 55…
3 E92000001 England          Lloegr           E92000001 England                     2 (((0.7817 52.98, 0.7865 52.9791, 0.7883 52.9793…
4 W92000004 Wales            Cymru            W92000004 Wales                       3 (((-2.7107 51.5696, -2.7228 51.5688, -2.7235 51…
>
```

FIGURE 27 GEOCODE LABELS (OWN PRODUCTION)

We can see now how we have the geocode, the labels, the names and the geometries are well represented so now we can use this data to draw boundaries in our map.

ELECTRICITY CONSUMPTION

Since the data was spread across different tabs, we have now to join all the tabs and recode the columns let's check again the structure.



Subnational electricity consumption, Great Britain, 2007

This table shows the electricity consumption and the number of electric meters by local authority. Country/region totals are shown where the local authority is specified as "All local authorities".
Some consumption within given countries/regions could not be allocated to specific local authorities. This consumption is shown in the rows where "Unallocated" is specified in the local authority column.
Freeze panes are active on this sheet. To turn off freeze panes select the 'View' ribbon then 'Freeze Panes' then 'Unfreeze Panes' or use [Alt W, F].

Code	Country or region	Local authority	Number of meters (thousands): Domestic	Number of meters (thousands): Non-Domestic	Number of meters (thousands): All meters	Total consumption (GWh): Domestic	Total consumption (GWh): Non-Domestic
	England	**All local authorities**	**22,613.2**	**2,070.8**	**24,684.1**	**99,598**	**161,667**
UKC	North East	All local authorities	1,184.2	82.0	1,266.2	4,430	8,912
UKD	North West	All local authorities	3,108.9	243.4	3,352.3	13,139	22,214
UKE	Yorkshire and The Humber	All local authorities	2,288.1	178.9	2,477.0	9,377	16,083
UKF	East Midlands	All local authorities	1,957.1	161.3	2,118.4	8,518	14,120
UKG	West Midlands	All local authorities	2,347.9	199.7	2,547.6	10,407	16,568
UKH	East	All local authorities	2,488.8	217.4	2,706.2	11,934	16,324
UKI	London	All local authorities	3,310.0	412.5	3,722.5	13,774	28,423
UKJ	South East	All local authorities	3,581.4	333.0	3,914.4	16,981	23,806
UKK	South West	All local authorities	2,336.7	242.7	2,579.4	11,039	15,228
UKL	**Wales**	**All local authorities**	**1,303.0**	**124.2**	**1,427.2**	**5,399**	**11,234**
UKM	**Scotland**	**All local authorities**	**2,721.0**	**224.8**	**2,945.8**	**12,001**	**16,961**
	Unallocated	All local authorities	33.1	15.0	48.1	128	2,681
UKC2101	North East	Alnwick	16.8	1.9	18.7	76	75
UKC2102	North East	Berwick-upon-Tweed	16.0	2.4	18.4	81	98
UKC2103	North East	Blyth Valley	37.0	2.0	39.0	134	175
UKC2104	North East	Castle Morpeth	22.5	2.0	24.4	103	110
UKC1409	North East	Chester-le-Street	24.7	1.3	26.0	89	71
UKC1300	North East	Darlington	48.7	3.6	52.3	188	304
UKC1403	North East	Derwentside	41.9	2.5	44.3	154	172
UKC1410	North East	Durham	38.8	2.9	41.8	142	283
UKC1411	North East	Easington	43.3	2.4	45.8	153	270
UKC2201	North East	Gateshead	92.9	6.4	99.3	331	669
UKC1101	North East	Hartlepool	41.7	2.9	44.6	151	355
UKC1201	North East	Middlesbrough	60.6	4.2	64.8	225	616
UKC2202	North East	Newcastle upon Tyne	125.0	9.9	134.9	476	909
UKC2203	North East	North Tyneside	94.9	5.6	100.5	343	615
UKC1202	North East	Redcar and Cleveland	62.7	3.7	66.4	238	975
UKC1406	North East	Sedgefield	41.0	2.5	43.5	147	314
UKC2204	North East	South Tyneside	69.5	4.0	73.4	236	247

FIGURE 28 ELETRICITY CONSUMPTION BY REGION (OWN PRODUCTION)

Let's load the library 'readxl' to read excels files.

```
library(readxl)
```

Our data is spread across 15 tabs only the last 13 contain all the values we need from 2015 so we will use the function 'read_excel' over a loop to read all the sheets we need skipping the first 4 lines

```
t <- 2:15
l=list()
for (i in seq_along(t))
  {
l[[i]] <- read_excel("data/subnational_electricity_consumption_statistics_2005-2020.xlsx",
           sheet = t[i], skip = 4)
  }
```

Now for every sheet, we read we create new columns called year starting from 2005 on.

```
#adding year columns
t = list()
for (i in seq_along(l)){
  t[[i]]=l[[i]]%>%
    mutate(year = 2005+i-1)
}
```

The next step is to create a single table combining all the sheets by rows and selecting the relevant variables:

- Code
- Country or Region
- Local authority
- Total consumption (Gwh)
- year

```
#adding all rows
t[[2]]%>%
  select(year)
rbind(l)
t <- bind_rows(t)
write_csv(elect_final_consumption_per_terrti,'electricity_consumption_region.csv')
```

Let’s check the result.

electricity_consumption_region

Code	Country or region	Local authority	Total consumption (GWh):	year
NA	Great Britain	All local authorities	320314.37070608100	2005
NA	England	All local authorities	267948.90052630000	2005
UKC	North East	All local authorities	13897.0204194	2005
UKD	North West	All local authorities	36292.02624819180	2005
UKE	Yorkshire and The Humber	All local authorities	26835.621181	2005
UKF	East Midlands	All local authorities	23938.219754990400	2005
UKG	West Midlands	All local authorities	27253.86121611920	2005
UKH	East	All local authorities	29273.097626764400	2005
UKI	London	All local authorities	41434.85720787400	2005
UKJ	South East	All local authorities	41757.38086976010	2005
UKK	South West	All local authorities	27266.8160022	2005
UKL	Wales	All local authorities	17567.14764925930	2005
UKM	Scotland	All local authorities	29505.321898020600	2005
NA	Unallocated	All local authorities	5293.000632501360	2005
UKC2101	North East	Alnwick	164.62184180000000	2005
UKC2102	North East	Berwick-upon-Tweed	183.87219600000000	2005
UKC2103	North East	Blyth Valley	337.1297446000000	2005
UKC2104	North East	Castle Morpeth	237.40340690000000	2005
UKC1409	North East	Chester-le-Street	166.50976060000000	2005
UKC1300	North East	Darlington	510.4431921000000	2005
UKC1403	North East	Derwentside	337.194379	2005
UKC1410	North East	Durham	451.3188705	2005
UKC1411	North East	Easington	423.07154840000000	2005
UKC2201	North East	Gateshead	1009.328399	2005
UKC1101	North East	Hartlepool	549.3147793	2005
UKC1201	North East	Middlesbrough	797.8128884	2005
UKC2202	North East	Newcastle upon Tyne	1433.2105155	2005

FIGURE 29 ELECTRICITY CONSUMPTION (OWN PRODUCTION)

We have cleaned electricity consumption and now is ready for production.

CO2 EMISSION

CO2 emission data comes already in a desirable format the only thing we need to do is to select the variables, filter by rows selecting CO2 gas emission and transport sector also group them by countries compute the average emission by country and standardise the date

```
uk_local_authority <- read_csv("/Users/eduardromero/Library/Mobile Documents/com~apple~CloudDocs/Documents/London
Metropolitan
University/Msc_production_project/data_processing/data_processing/data/uk-local-authority-ghg-emissions-2020-datase
t.csv")
#filtering co2 data
uk_local_authority <- uk_local_authority%>%filter(Country!='Unallocated',
                           `Greenhouse gas`=='CO2',
                           `LA GHG Sector`=='Transport')
uk_local_authority

#stadarizing the date
uk_local_authority <- uk_local_authority%>%
  mutate(`Calendar Year` = as.Date(paste(`Calendar Year`,'01','01',sep = '-')))%>%
  rename('date' = `Calendar Year`)%>%
  select(date,everything())

#Grouping by countries computing the mean
 uk_local_authority <- uk_local_authority%>%
  group_by(Country,date)%>%
  summarise(mean_co2=mean(`Territorial emissions (kt CO2e)`))
 uk_local_authority_plot
```

```
> uk_local_authority_plot
# A tibble: 64 x 3
# Groups:   Country [4]
   Country date       mean_co2
   <chr>   <date>        <dbl>
 1 England 2005-01-01     84.3
 2 England 2006-01-01     83.2
 3 England 2007-01-01     83.8
 4 England 2008-01-01     80.1
 5 England 2009-01-01     77.6
 6 England 2010-01-01     77.0
 7 England 2011-01-01     75.8
 8 England 2012-01-01     74.8
 9 England 2013-01-01     74.1
10 England 2014-01-01     75.0
# … with 54 more rows
```

FIGURE 30 GREENHOUSE EMISSION (OWN PRODUCTION)

CHARGE POINT DATA

Let's now clean the charge point data. In this case, we will carry out several operations in order to work with this dataset since this is the main dataset for the App. In the first place, we will use an auxiliary dataset that contains the postcode in the UK and many other geographic features like county, country, or city. The reason why we do this is that we need to transform Charge point data into a map object in order to work with this data it means we need to set up the geometries since the charge point dataset contains the postcodes, we can use this column to join it with another dataset that contains maps features, in this case, the "NSPL_FEB_2021_UK.csv" which contains all the regional level in the UK.

The postcode data can be obtained at
https://geoportal.statistics.gov.uk/datasets/7606baba633d4bbca3f2510ab78acf61/about

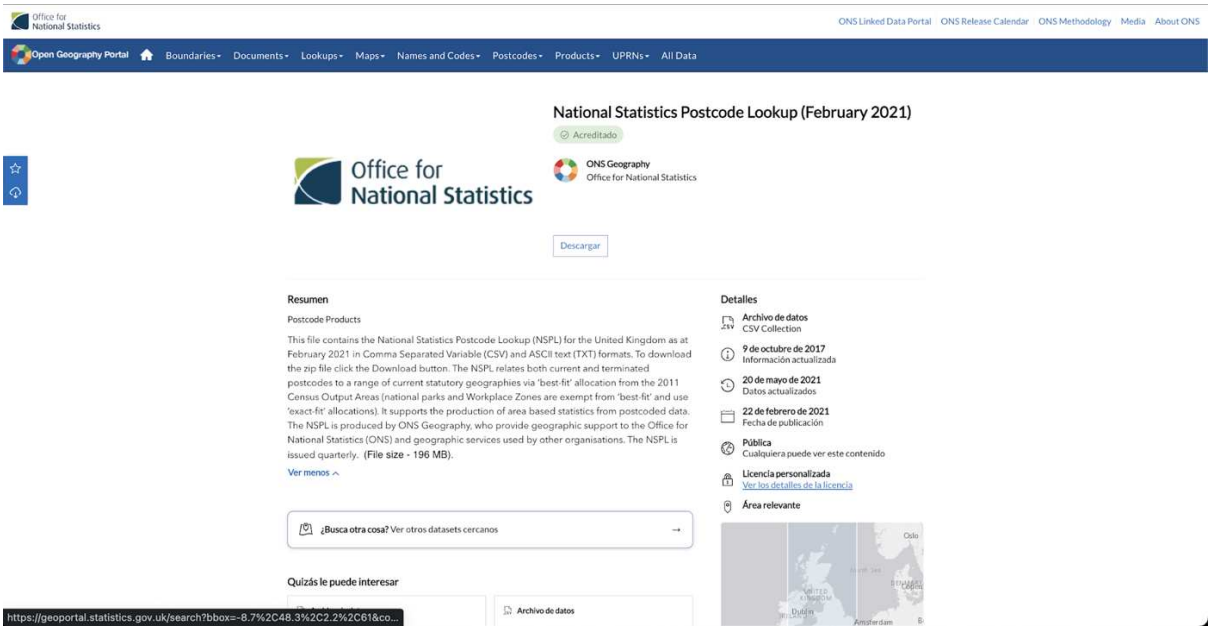


FIGURE 31 HTTPS://GEOPORTAL.STATISTICS.GOV.UK/DATASETS/7606BABA633D4BBCA3F2510AB78ACF61/ABOUT

The file that can be downloaded contains the main dataset and the look-up tables for each level region. To proceed with this data, we load the whole dataset 'NSPL_FEB_2021_UK.csv' and also the look-up table for countries that come with the files 'Country names and codes the UK as at 08_12.csv'

Once we load the tables, we just join them making a left join by postcode

```
postcode <- read_csv("/Users/eduardromero/Library/Mobile Documents/com~apple~CloudDocs/Documents/London Metropolitan
University/Msc_production_project/data_processing/data_processing/data/NSPL_FEB_2021_UK/Data/NSPL_FEB_2021_UK.csv",
)
lookup_crty <- read_csv("/Users/eduardromero/Library/Mobile Documents/com~apple~CloudDocs/Documents/London Metropolitan
University/Msc_production_project/data_processing/data_processing/data/NSPL_FEB_2021_UK/Documents/Country names and
codes UK as at 08_12.csv")

charge_point = charge_point%>%
  left_join(postcode,by = c('postcode'='pcd'))
```

Once we join it by postcode the next step is to use the look-up table for the country to add the country column to the charge point dataset

```
charge_point = charge_point%>%left_join(lookup_crty,by = c("ctry" ="CTRY12CD" ))
```

Now we can join the charge point with a map object since we have now a country column

```
charge_point=charge_point %>%left_join(uk_json,by =c("CTRY12NM" ='geo_label') )
```

Having the country column and the geometries we can now select the relevant variables:

- Latitude
- Longitude
- buildingName
- street
- date
- geometry

```
charge_point <- charge_point%>%select(name.x,
                      latitude,
                      longitude,
                      buildingName,
                      street,
                      date,
                      geometry)
```

Finally, we set it up as a map object.

```
charge_point= charge_point%>%st_as_sf()
```

CONCLUSION

Cleaning data is estimated to take around 60-70% of the time of a data analyst. Here we see that the process wasn't trivial in some cases, and it could even take longer if some steps were realised manually like joining different sheets in excel. In order to manage all these challenges, a good understanding of the data analyst tools has become crucial.

CREATING THE R SHINY TEMPLATE

INTRODUCTION

In this section, we will develop the template necessary for our app. The template will consist of a screen with two tabs one for the plots I will develop and one for the map itself. The map will contain two layers one for the location of the charge point and another one for the CO2 emission data. The charge point location will be represented by markers on the map meanwhile CO2 emission level will be represented by a gradient of colours.

SHINY STRUCTURE

The shiny structure is composed of two blocks. The first block contains the necessary code to represent the visual structure for the users. In this part, we can define the windows, the cards, the colours, the borders, the font, the font colours, etc.

Let's define the elements necessary to develop the structure. In the first place, we will define our project under bootstrap technology. Bootstrap is a framework that gathers all the predefined CSS elements necessary for the website this way we do not create all the elements from the beginning we just need to call it. Bootstrap includes web responsive technology to adapt to every size of the screen. In r the code looks this way.

```
ui <- bootstrapPage()
```

Once we define under what framework we want to work we can now define the visual set for the screen. For that, we introduce CSS code under the bootstrapPage section. As we said, we will add a card under the map windows. This card will contain some plots and a slider bottom that will contain the date. The card will have an id called controls and as we are in a bootstrap scenario, we can also add an event when the mouse hovers on the card, for example, the card may be slightly transparent, but when you hover over it, it becomes opaque. Let's see how it goes

```
ui <- bootstrapPage(

  tags$style("
```

```
        #controls {
          background-color: #fff;
          opacity: 0.5;
        }
        #controls:hover{
          opacity: 1;
        }
              "))
```

Checking at the previous code we see that for each element with id ‘controls’ the background colour will be white ‘#fff’ and its initial opacity is 0.5. Then we add one more property for the id ‘controls’ which is called hover, it means once we hover over the element with id ‘controls’ its opacity turns into 1.

Our screen will compose of two tabs. To define the tab, we use the keyword navbarPage to define the name of the navigation bar. In this case, it will be UK 2040 zero emission. The first tab will be called map while the second will be called plot.

```
ui <- bootstrapPage(

  tags$style("
        #controls {
          background-color: #fff;
          opacity: 0.5;
        }
        #controls:hover{
          opacity: 1;
        }
              "),
  navbarPage("UK 2040 zero emission",
             tabPanel("Map"
                      ),
         tabPanel('Plots')
  )
)
```

In the map tab, we will add the card that will contain the plots and the slider to browse along the dates. The card will have the id ‘controls’ and it will heredity all properties defined in the tags$style we defined before. The card is called with keyword absolutPanel and it contains four elements four plots, and one slider. The plots will be the cumulative charge point in, the UK and the total emission of CO2 per country in the UK, the type of electric vehicles and electricity supply. Finally, the slider will contain the date.

The date in the slider input will be the main feature in this screen since the slider controls the map and the plots if we slide the bottom, the map and the plot will change showing the status of the data at that particular point in time.

In this particular example, for the sliderInput we set the choices in a vector that contains the date we want to run along, for example, choices = `c('2001-01-01','2002-01-01','2003-01-01','2004-01-01')` and the starting point,`selected = '2004-01-01'`.

```
ui <- bootstrapPage(

  tags$style("
        #controls {
          background-color: #fff;
          opacity: 0.5;
        }
        #controls:hover{
          opacity: 1;
        }
              "),
  navbarPage("UK 2030 zero emission",
```

```
            tabPanel("Map",
                     absolutePanel(id = "controls", class = "panel panel-default",
                                   top = 75, left = 55, width = 450, fixed=TRUE,
                                   draggable = TRUE, height = "auto",
                                   plotOutput("plot_cp",
                                              height="130px",
                                              width="100%"),
                                   plotOutput("cumulative_plot_co2",
                                              height="230px",
                                              width="100%"),
                                   sliderTextInput("plot_date",
                                                   label = h5("Select the date"),
                                                   choices = c('2001-01-01','2002-01-01','2003-01-01',
                                                               '2004-01-01'),
                                                   selected = '2004-01-01',
                                                   grid = FALSE,
                                                   animate=animationOptions(interval = 5000, loop = FALSE)),

                     ),
            tabPanel('Plots')
  )
))
```

Let's check now the structure.

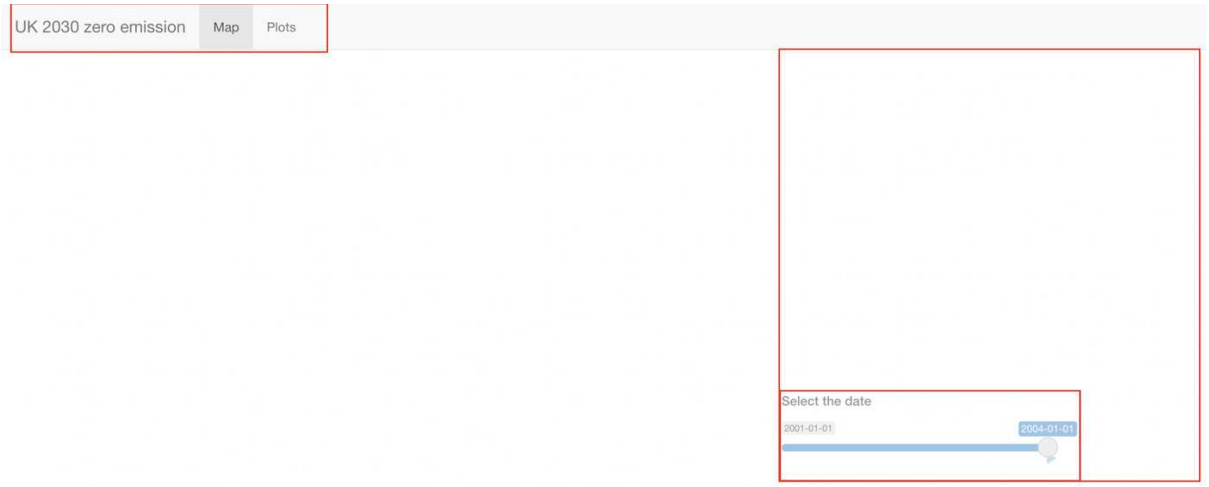


FIGURE 32 TEMPLATE (OWN PRODUCTION)

The second tab 'Plots' will be composed of some set of panels orientated to choose one data set between two: Electric vehicles and electricity consumption. The user can choose between the territories and the date. In the case of an electric vehicle, there will be available monthly data, quarter, and annual periods. the structure is very simple.

```
selectInput('territory', 'Territory',
              choices = unique(ulev_clean$Geography)),
  selectInput('period', 'Period', choices = NULL),
  selectInput('units', 'Units', choices = NULL),
  tableOutput('data'),
  plotOutput('date')
```



FIGURE 33 PLOTS TAB (OWN PRODUCTION)

DEVELOPING THE VISUAL STRUCTURES

Once we have done the template is time now to create the visual structures, which means, the maps and the plots. In the first place, we will create the map. The map will contain the charge point location and some features related to CO2 emission.

The map will be built using the Leaflet library. The initial map will be called the base since from this map all the features will be developed. In first the first place, we called the leaflet object then we call a Raster Type object in this case 'addTiles()' which will load a map with simple features in blue and white

```
basemap = leaflet()%>%
  addTiles()%>%
```



FIGURE 34 ADDTILES OBJECT(OWN PRODUCTION)

Once we define the Raster Type we define how many features we want to include in the map. In this case, two main features will be included to know, CO2 emission and charge point location. Since these two features will be added through layers we use the grouped types 'addLayersControl()'. These features add two checklists to the map that let us choose when to visualize any layer. Each group will be named 'Location charge points' and 'CO2 Emission' and we will choose the option collapsed = F. This way the option will be clearly seen

```
basemap = leaflet()%>%
  addTiles()%>%
addLayersControl(
    position = 'bottomright',
    overlayGroups = c('location charge points', 'Co2 Emission'),
    options = layersControlOptions(collapsed=F))
```



FIGURE 35 ADDING LAYERS (OWN PRODUCTION)

Once we add the layers, we also add the legends. As we haven't added the data yet, we will leave it to show it later.

Till now we have defined the datasets, the template, and the visual structure. It is time now to put everything together and that will be possible thanks to the second part of the Shiny library which is called 'server', which can be thought of as the backend area. The server function will include all the necessary data to turn on the application together with all kinds of input from the user and other programs.

Let's start setting and formatting the input from the card 'absolutePanel()' created before. On that card, we had a slider input that indicate the date. Each step is actually an input that goes to the server and updates the dataset, automatically creating a filter over the data to reproduce the scenario at that time.

This part is a bit difficult to knit. To carry out this part we need to consider what elements will react once the user slides the bottom. In this case, the elements that depend on that slides are:

- The plots:
    - Cummulative_plot_cp
    - Plot_co2
    - Electric vehicle
    - Electricity
- The map

Since the slider input has the ‘plot_date’ as id we will refer to this object from the server area with its identifier ‘plot_date’. For example, the keyword ‘sliderTextInput’ contains the identifier ‘input’ in its name so from the server side we should get the input and store it somewhere

```
server <- function(input,output,session){
  formatted_date = reactive({
    format(as.Date(input$plot_date))
```

Checking the code we see how the server function has three arguments: input, output and session. Each function in the user interface that contains the identifier input in its name should have a receptor on the server side in this case ‘input$plot_date’ receive this input. Once we receive the input, we format the input as a date and store it in ‘formatted_date’. The keyword ‘reactive’ encapsulates the item in such a way that only direct callers can access it avoiding the possibility to change the input.

Once we receive the inputs, we need to link the input to the datasets that will be affected by it. That will be the ‘charge point’ dataset, CO2 emission dataset, electric vehicle, and electricity data. The relationship will be based on a filter function. Once the user slides the bottom automatically the filter function is activated applying a filter in the datasets showing all the datasets previous to that point.

```
server <- function(input,output,session){
  formatted_date = reactive({
    format(as.Date(input$plot_date))
  })
  reactive_cp = reactive({
    charge_point%>% filter(date <= formatted_date())
  })
  reactive_db_co2 = reactive({
    uk_local_authority_emission%>% filter(date <= formatted_date())
  })[…]
```

The third element is the map. Having loaded the data, we can visualize the map adding more features like legends and colours but first, we need to integrate our map into the server side. As we said before each input needs to have a receptor on the server side but also the server needs to send back the result and for that, we need to also get a receptor in the user area. In this case, if we want to deploy the map on the user area, we need the keyword ‘leafletoutput()’ in the tabpanel map in our previous visual Template.

```
navbarPage("UK 2040 Zero-emission",
             tabPanel("Plot",
                      leafletOutput('mymap',
                                    width = '800px',
                                    height = '800px'),
                      absolutePanel(id = "controls", class = "panel panel-default",
                                    top = 75, left = 55, width = 450, fixed=TRUE,
                                    draggable = TRUE, height = "auto",
                                    plotOutput("cumulative_plot_cp",
                                               height="130px",
                                               width="100%"),
                                    plotOutput("cumulative_plot_co2",
                                               height="230px",
                                               width="100%"),
                                    sliderTextInput("plot_date",
                                                    label = h5("Select the date"),
                                                    choices = unique(charge_point$date%>%na.omit()),
                                                    selected = current_date,
                                                    grid = FALSE,
                                                    animate=animationOptions(interval = 5000, loop = FALSE)),

                      )
  )
```

Once we add the ‘leafletoutput()’ we are ready to send the map we previously received. But first, we need to set up the map for each change that is produced on the slide bottom. It means we need to add features from the datasets.

In the first place, we use the ‘observeEvent()’ function. This function makes it possible to change all the set-up for every single input in the map and sent it back with new changes. The function contains two arguments, the first argument is the element that will be used to change the structure of the map ‘input$plot_date’, the second, all the code that will receive an update due to the new input which is going to be the map. In the map, we will add new features like the circles that indicate where the charge point is grouped. Each circle contains a number that indicates the total charge point in that area. If click on it, we can zoom in and see every single charge point.

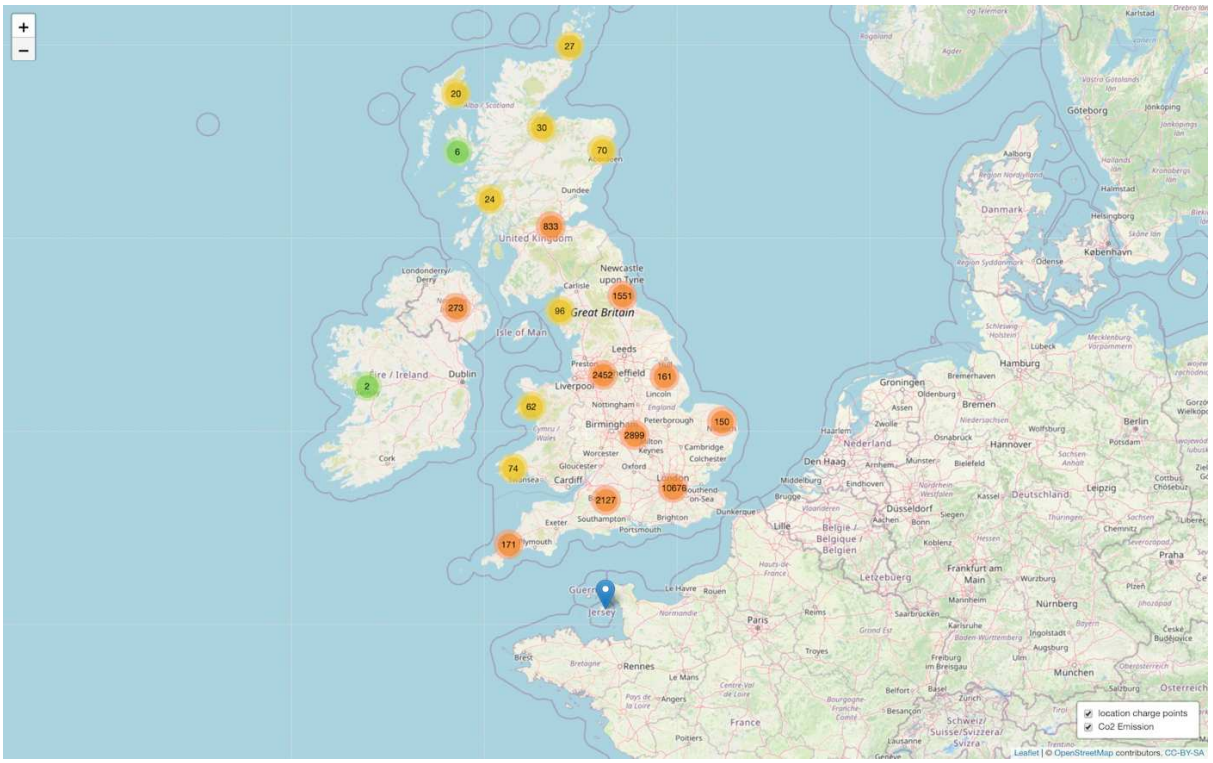


FIGURE 36 CHARGE POINTS (OWN PRODUCTION)

We add this feature with the function addMarkers(). The function needs the data, the latitude and longitude of every device and the group of layers it belongs, finally some labels can be added to every single marker. Since we want to make a group of circles when we zoom in or out we add the parameter ‘clusterOptions = markerClusterOptions();.

```
observeEvent(input$plot_date,{
    leafletProxy('mymap') %>%
      clearMarkers()%>%
      clearMarkerClusters()%>%
      addMarkers(data = reactive_cp(),
                 lat = ~latitude,
                 lng = ~longitude,
                 group = 'location charge points',
                 label = sprintf("<strong>Name:</strong> %s<br/>
                                 <strong>Street</strong>: %s",
                                   reactive_cp()$name.x,
                                   reactive_cp()$street)%>%
                   lapply(htmltools::HTML),
                 clusterOptions = markerClusterOptions())
```

The second feature to add will be the legend and the map for CO2 data. For that we add the boundaries, the group to ‘Co2 Emission’ we fill the colour with the blue palette with a gradient depending on the interval of emission, in this case, I divided by percentiles using the quantile

function ‘quantile()’. The value to get quantiles is the ‘mean_co2’. Once we set the quantile we use the ‘colorBin()’ function to define the pallete chosen for the map, in this case, Blue.

```
#coloring the map
bins=round(quantile(uk_local_authority$mean_co2, probs = seq(0, 1, 0.2)),0)%>%as.vector()


cv_pal <- colorBin("Blues", domain = uk_local_authority$mean_co2, bins = bins)


observeEvent(input$plot_date,{
    leafletProxy('mymap') %>%
      clearMarkers()%>%
      clearMarkerClusters()%>%
      addMarkers(data = reactive_cp(),
                 lat = ~latitude,
                 lng = ~longitude,
                 group = 'location charge points',
                 label = sprintf("<strong>Name:</strong> %s<br/>
                                 <strong>Street</strong>: %s",
                                   reactive_cp()$name.x,
                                   reactive_cp()$street)%>%
                   lapply(htmltools::HTML),
                 clusterOptions = markerClusterOptions())
      addPolygons(data =reactive_db_co2(),
                  group = 'Co2 Emission',
                  fillColor = ~cv_pal(reactive_db_co2()$mean_co2),
                  stroke = FALSE, smoothFactor = 0.2, fillOpacity =.1)


  })
```

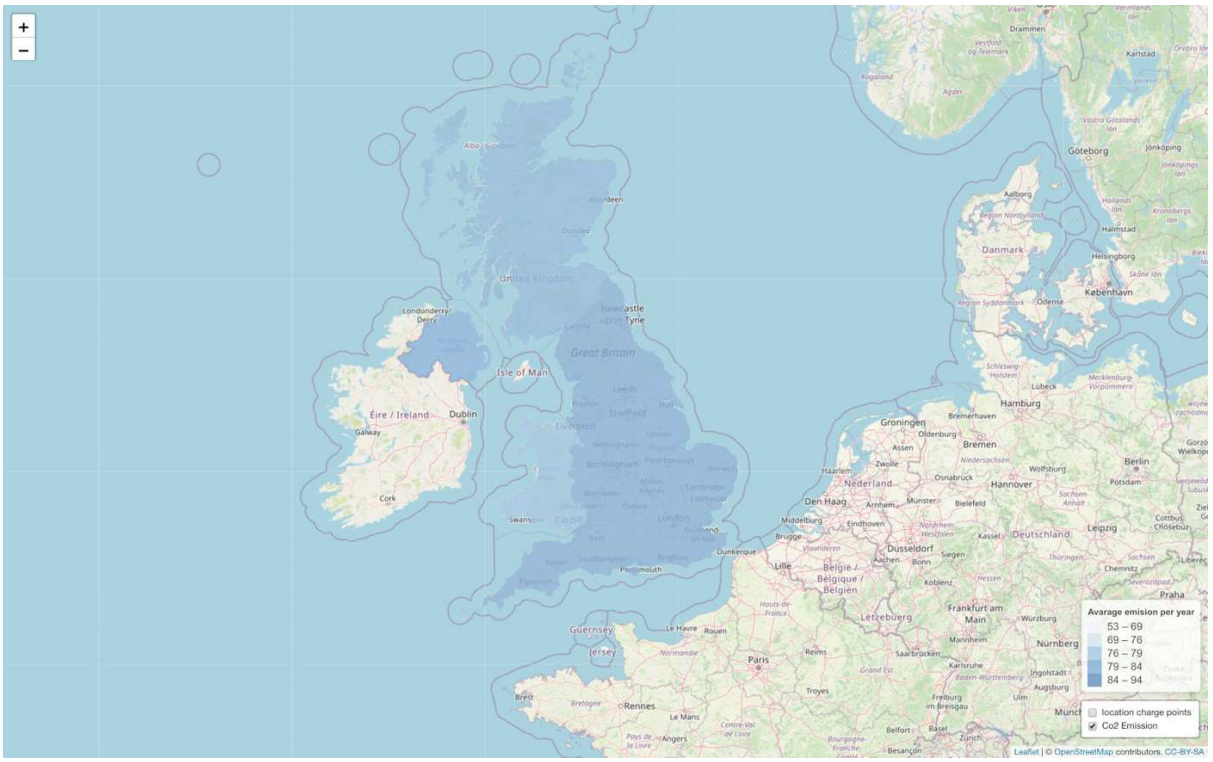


FIGURE 37 CO2 EMISSION BY COLOR (OWN PRODUCTION)

Each feature can be activated or deactivated using the checklist bottom in the right bottom.

Once the map has been done, the next step now is to build the card and attach the plots. As we said before the plot depends on the slide input. To make the dependency explicit we need to write the

plot in terms of the ‘formatted_date()’. For that, we use the ‘ggplot()’ function filtering the data in the function of ‘formated_date()’. The first plot will be the total charge group by date, but before we need to sum all the charge points for every year. We will do it in different steps:

- We set ‘dateCreated’ variable to date
- We format ‘dateCreated’ to %Y-01-01 this way all the date from the different dataset is homogenous.
- We rename the variable ‘dateCreated’ to ‘date’ to make it homogenous
- We group by date summing every observation and we call it ‘new_by_year’
- We add a new column ‘csum’ that will accumulate all the new cases
- Finally, we plot the new table

```
#unique date %Y-01-01

charge_point = charge_point%>%
  mutate(dateCreated = as.Date(dateCreated),
         dateCreated = format(dateCreated,'%Y-01-01'))%>%
  rename('date' = 'dateCreated')
#cumulative_data_plot charge point
  new_by_year =charge_point%>%
  group_by(date)%>%
  summarise (new_by_year = n())%>%na.omit()


new_by_year$csum <- ave(new_by_year$new_by_year,FUN = cumsum)
charge_point_aggregated <- new_by_year%>%
  mutate(date = as.Date(date))
```

To add the new plot to the app, we have to have a plot output on the user side, and this is

```
plotOutput("cumulative_plot_cp",
                                              height="130px",
                                              width="100%")
```

From the server side, we send the plot wrapping the plot inside the ‘renderPlot()’ function

```
output$cumulative_plot_cp <- renderPlot(
   {
     charge_point_aggregated%>%
       filter(date<formatted_date())%>%
       ggplot(aes(date,csum))+
       geom_line() + geom_point(size = 1, alpha = 0.8) +
       ylab("Cumulative") + xlab("Date") + theme_bw()
   })
```

The same operation has to be carried out for the CO2 emission plot and so on

```
output$cumulative_plot_co2 <- renderPlot(

     uk_local_authority_plot%>%filter(date<formatted_date())%>%
       ggplot(aes(date, mean_co2,color = Country))+
       geom_line() + geom_point(size = 1, alpha = 0.8) +
       ylab("Cumulative") + xlab("Date") + theme_bw()
```

Let’s check the results.

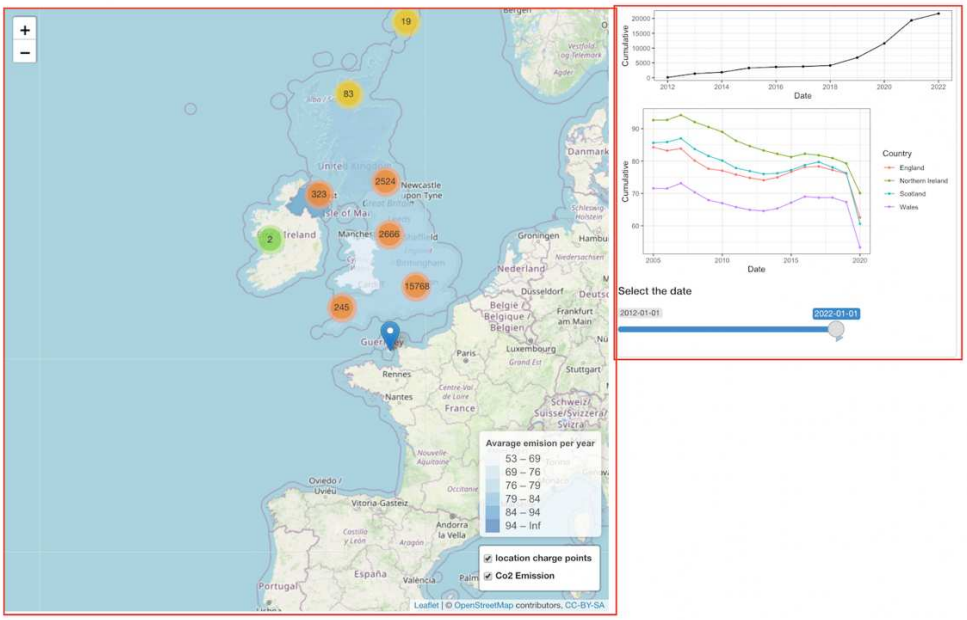


FIGURE 38 SAMPLE MAP(OWN PRODUCTION)

Now we can appreciate the plots the map and the legend. Is the time now to develop the plots in the other tab. The data to be analysed will be electric vehicles and electric consumption. For that, we load first the data.

```
library(tidyverse)
library(shiny)
ulev_clean <- read_csv("/Users/eduardromero/Library/Mobile Documents/com~apple~CloudDocs/Documents/London
Metropolitan University/Msc_production_project/data_processing/data_processing/data/ulev_clean.csv")
```

Once we load the data, we filter the data by excluding all the types marked as a total since this is a row that gathers several rows in one. The structure for the user side has been already explained. Let's see the previous code.

```
ui <- bootsTrapPage(
  selectInput('territory', 'Territory',
              choices = unique(ulev_clean$Geography)),
  selectInput('period', 'Period', choices = NULL),
  selectInput('units', 'Units', choices = NULL),
  tableOutput('data'),
  plotOutput('date')
)
```

We have to create the receptor for the first input 'territory'. The receptor will be composed of the data, in this case, electric vehicle ('ulev_clean') filtering the geography in the function of the user input 'input$territory'. Once the user introduces the territory, the data will change automatically filtering the desired input.

```
server <- function(input,output, session){
  territory <- reactive({
    filter(ulev_clean,
           Geography == input$territory)
  })
```

Once the input has been introduced, the server has to create a copy of the dataset with the filter applied and now the server has to update the next input available under that filter. For that, shiny use the function 'observeEvent()' to update the new choices in the dropdown selection.

```
server <- function(input,output, session){
  territory <- reactive({
    filter(ulev_clean,
           Geography == input$territory)
  })
  observeEvent(territory(),
               {
```

```
              choices <- unique(territory()$`Date Interval [note 3]`)
              updateSelectInput(inputId = 'period',
                                choices = choices)
            })
```

Once the period has been chosen, the server, again, has to update the dataset by applying the new filter, in this case, the period

```
server <- function(input,output, session){
  territory <- reactive({
    filter(ulev_clean,
           Geography == input$territory)
  })
  observeEvent(territory(),
               {
                 choices <- unique(territory()$`Date Interval [note 3]`)
                 updateSelectInput(inputId = 'period',
                                   choices = choices)

  #dado el territorio y period, obtengo los datos del period elegido
  periods <- reactive({
    req(input$period)
    filter(territory(),`Date Interval [note 3]`==input$period)
  })
```

Now we follow the same process to choose the units

```
server <- function(input,output, session){
  territory <- reactive({
    filter(ulev_clean,
           Geography == input$territory)
  })
  observeEvent(territory(),
               {
                 choices <- unique(territory()$`Date Interval [note 3]`)
                 updateSelectInput(inputId = 'period',
                                   choices = choices)

  #dado el territorio y period, obtengo los datos del period elegido
  periods <- reactive({
    req(input$period)
    filter(territory(),`Date Interval [note 3]`==input$period)
  })


  observeEvent(periods(),{
    choices <- unique(periods()$Units)
    updateSelectInput(inputId = 'units',choices = choices)
  })
```

When all the inputs have been applied is time now to send back the new changes. The receptor from the user side is ‘plotoutput()’ and ‘tableoutput()’ functions. Let’s start first with ‘tableoutput()’.

To send back the chosen table we have to use the function ‘renderTable()’ and introduce in the function the table we want to show, in this case, the electric vehicle which is inside the ‘periods()’ object with the filer applied.

```
  output$data <-
    renderTable({
      req(input$units)
      head(periods() %>%
             filter(Units == input$units))
    })
```

The second output is the plot. We store the object in the ‘output$date’ object and use the ‘renderPlot()’ function to insert the plot and send it back.

```
  output$data <-
    renderTable({
      req(input$units)
      head(periods() %>%
             filter(Units == input$units))
    })


  output$date <- renderPlot(
    periods()%>%
      filter(Units == 'Number',
             type != 'Total',
             `BodyType [note 2]`!='Total')%>%
      count(date, `BodyType [note 2]`,wt = count)%>%
      ggplot(aes(date,
                 log(n+1), color = `BodyType [note 2]`))+
```

```
      geom_line()+
      facet_wrap(~`BodyType [note 2]`)
  )

}
```

Let's see the result.



Geography	Date Interval [note 3]	Units	date	BodyType [note 2]	type	count
Great Britain	Monthly	Number	14610.00	Buses and coaches	Battery Electric	0.00
Great Britain	Monthly	Number	14610.00	Buses and coaches	Plug-in hybrid electric	0.00
Great Britain	Monthly	Number	14610.00	Buses and coaches	Range extended electric	0.00
Great Britain	Monthly	Number	14610.00	Buses and coaches	Hybrid electric	0.00
Great Britain	Monthly	Number	14610.00	Buses and coaches	Fuel cell electric	0.00
Great Britain	Monthly	Number	14610.00	Buses and coaches	Other fuels	0.00

FIGURE 39 SAMPLE OF PLOT TAB(OWN PRODUCTION)

Here we can clearly see the selector and the outputs. The same operation can be carried out with the demand electricity dataset

DEPLOYING THE APP

Is the time now to deploy the full App. To deploy the App we just need to run the app and press the button published this automatically will take

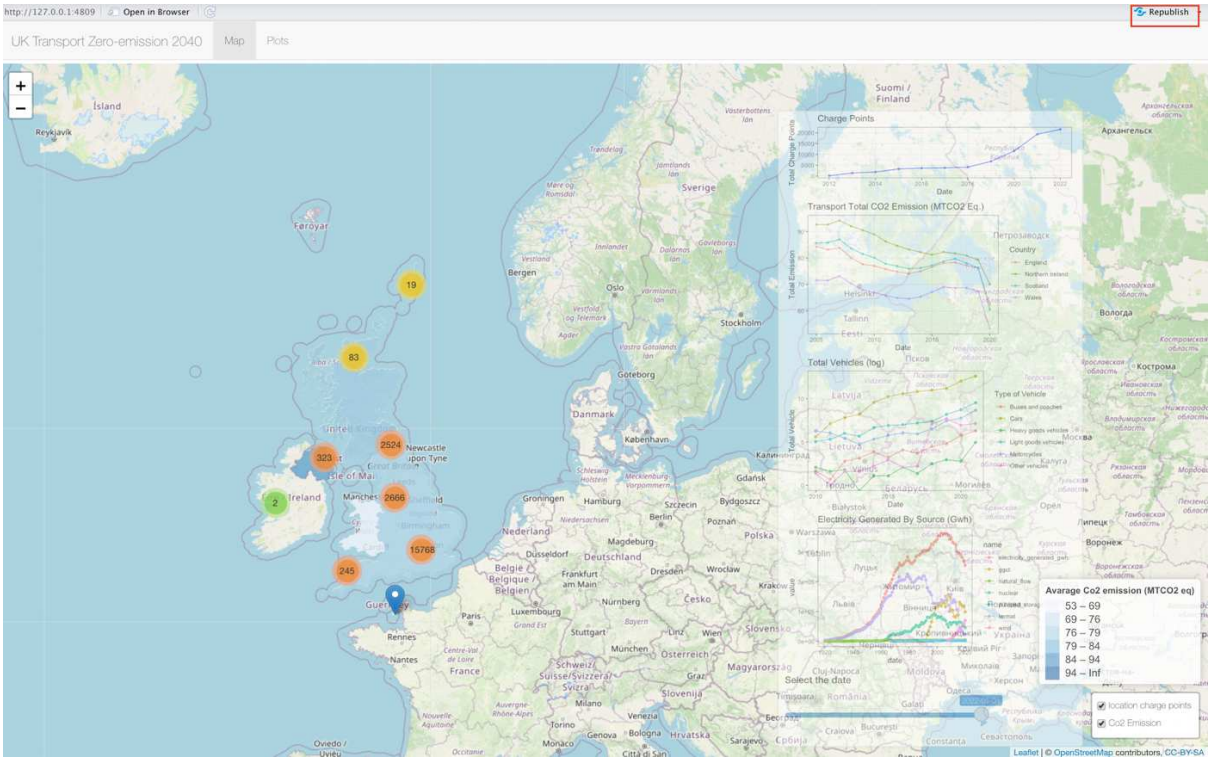


FIGURE 36 PUBLISHING THE APP(OWN PRODUCTION)

The app will be published on a server which belongs to RStudio. Once the App has been published, we can simply have access to it by its URL https://0d5iyx-eduard-romero0mercedes.shinyapp

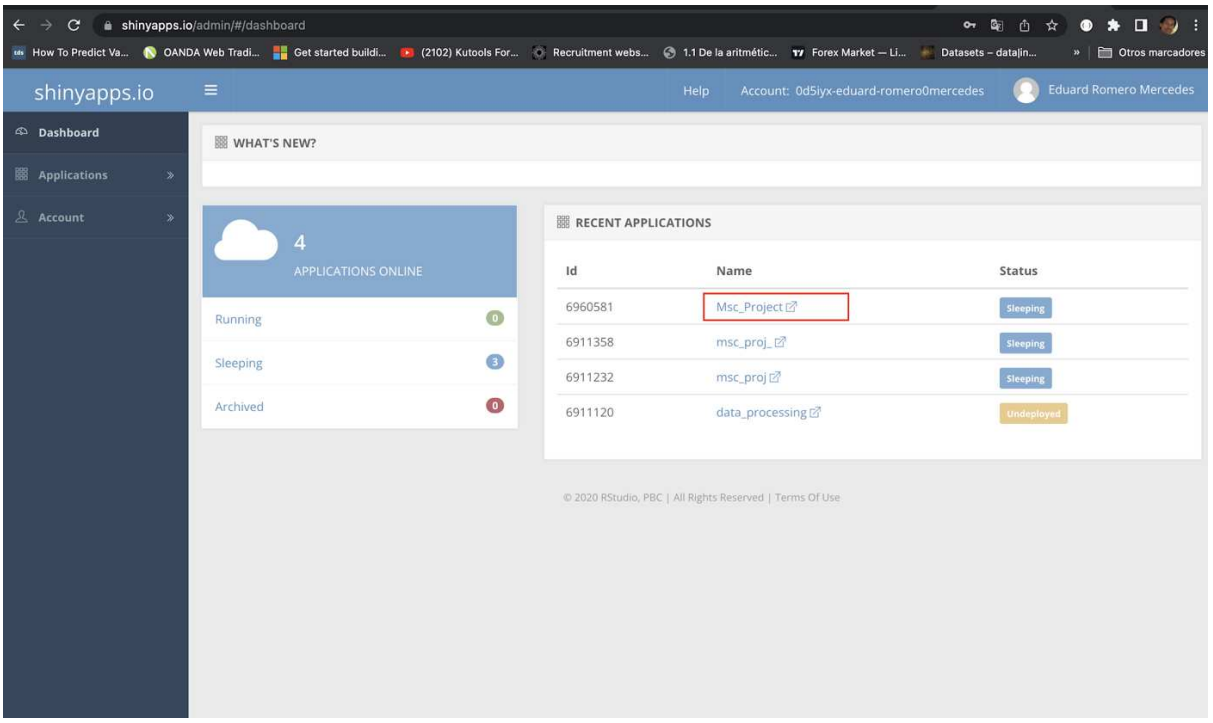


FIGURE 41 SERVER (OWN PRODUCTION)

On the shiny account the administrator can have access to a panel that monitories the traffic to the web application. Any user having provided the URL can have access to the app

CHAPTER 5: TESTING-RESULTS

The fully functional app is able to show every variable on the screen with its respective graph and location on the map when it is required. The app provided all the information needed to monitories the progress of every variable over time. If you click or hover over every single mark it shows the information of the charge point. Also, the slide bottom can get you to any point of time or if you want you can see the evolution.

The plot tab let you choose between two datasets, electricity demand and electric vehicles. This time electric vehicle can be split into several plot for each type of car and the period is an option which it can be monthly, annual, or quarter. For electricity demand, the period is only annual, but you can select every region to show the evolution of the variable for that particular region.

Let's test the App and go to the URL to visit it. If we type on the searching bar on Google, we can easily see the App

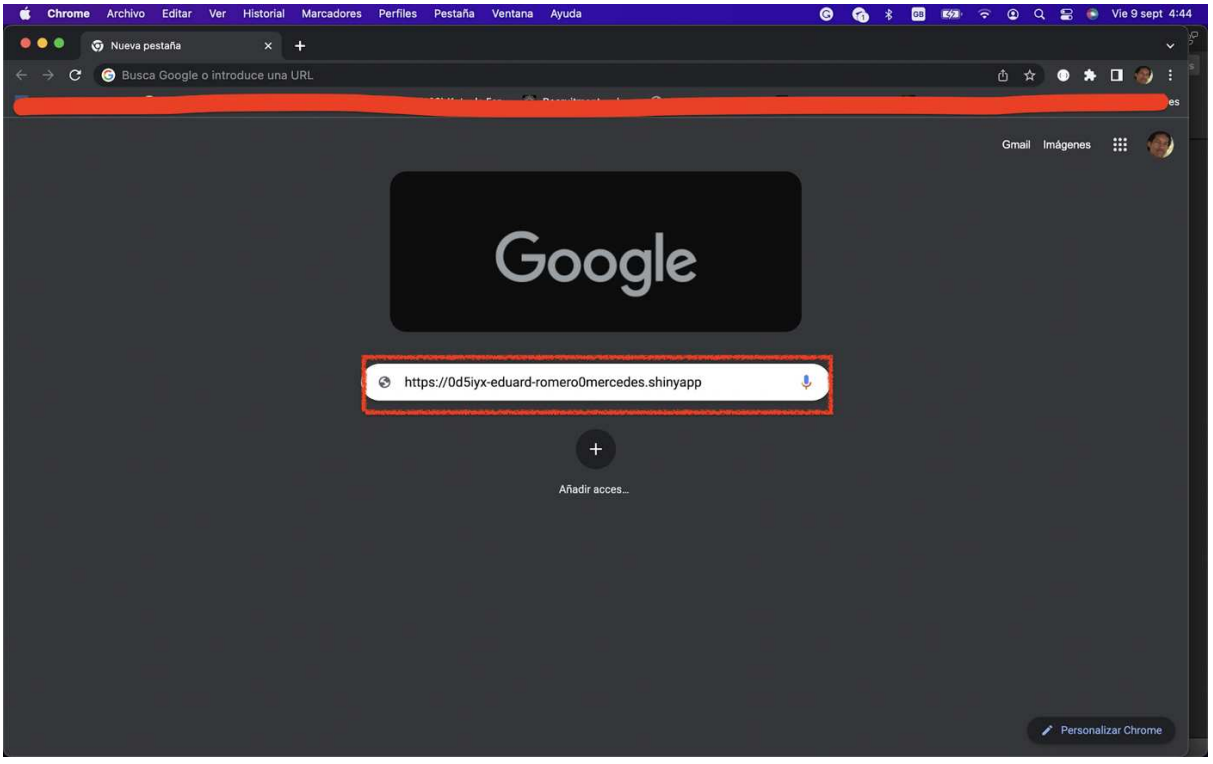


FIGURE 37 HAVING ACCESS (OWN PRODUCTION)

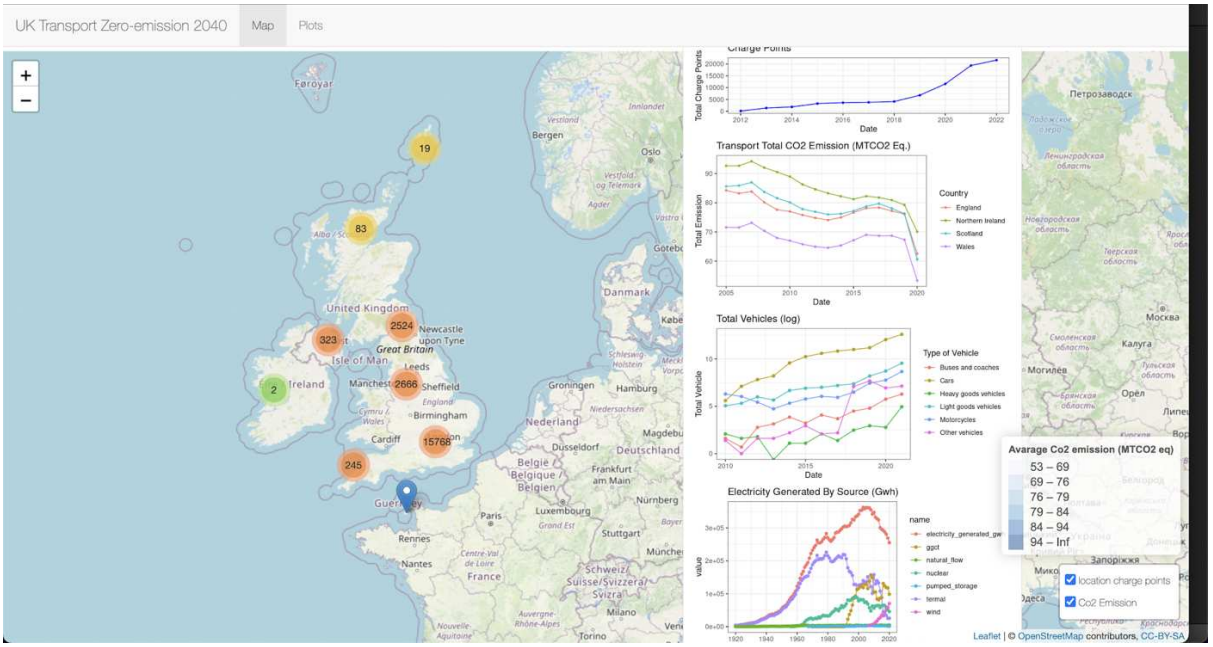


FIGURE 4 APP ON THE WEBSITE (OWN PRODUCTION)

Here we see the final result the App has been completely deployed.

CHAPTER 6: CONCLUSION

This work has shown the importance of the data and the new technologies to discover new knowledge coming from different data and resources. This reveals something very interesting and that is that today data is a fundamental and differentiating factor in all kinds of activities that can bring great benefits to those who analyse it or the interested parties but at the same time raises the alarm for the obvious conflict of interest that can arise if the source of the data is people.

Technologies such as machine learning in which large algorithms analyse data in the order of petabytes can disrupt people's lives as technology has always been doing, this in a positive or negative way. So, since the weapon does not kill until it is used, data analysis in good hands is a powerful weapon for the well-being of people, but if, on the other hand, the interests are opposed by those who decide to give their data, then said technology can be a very dangerous weapon.

It is essential that from the beginning there are rules and laws that protect the average citizen from this type of scenario since technology advances faster and faster. As a data analyst, it is my responsibility to treat the data correctly and respect the integrity, origin and privacy of the information and ensure that the data is always used not only to seek private benefit but also to serve society and that is what I wanted to do with this work

Beeley, C. (2016) *Web application development with R using shiny* -. 2nd ed. Birmingham, England: Packt Publishing.

Beeley, C. (2018) *Hands-On Dashboard Development with Shiny: A practical guide to building effective web applications and dashboards*. Birmingham, England: Packt Publishing.

Beeley, C. and Sukhdeve, S. R. (2018) *Web Application Development with R Using Shiny: Build stunning graphics and interactive data visualizations to deliver cutting-edge analytics, 3rd Edition*. 3rd ed. Birmingham, England: Packt Publishing.

Dalgaard, P. (2008) *Introductory statistics with R*. 2nd ed. New York, NY: Springer.

Davies, T. M. (2016) *The book of R: A first course in programming and statistics*. 1st ed. San Francisco, CA: No Starch Press.

Fay, C. *et al.* (2021) *Engineering Production-Grade Shiny Apps*. London, England: CRC Press.

Granjon, D. (2022) *Outstanding user interfaces with shiny*. London, England: CRC Press.

Grolemund, G. and Wickham, H. (2017) *R for Data Science*. Sebastopol, CA: O'Reilly Media.

Majumdar, P. (2021) *Learn emotion analysis with R: Perform sentiment assessments, extract emotions, and learn NLP techniques using R and shiny*. New Delhi, India: BPB Publications.

Sievert, C. (2020) *Interactive Web-Based Data Visualization with R, plotly, and shiny*. London, England: CRC Press.

Wickham, H. (2021) *Mastering shiny: Build interactive apps, reports, and dashboards powered by R*. Sebastopol, CA: O'Reilly Media.

CODE

```
library(shinyWidgets)
library(shiny)
library(rgdal)
library( 'spdplyr')
library(tidyverse)
library(leaflet)
library(sf)
library(readxl)
library(readxl)
library(lubridate)
library(rgdal)
if(!require(geojsonio)) install.packages("geojsonio", repos = "http://cran.us.r-project.org")

#Getting the map object to plot the map
uk_json <- geojson_read ("uk_country.geojson",what = 'sp')
#Passing the map object into an leaflet object and then into a tibble

uk_json<- uk_json%>%st_as_sf()
uk_json <- uk_json%>%tibble()

uk_json
#ELECTRICITY CONSUMPTIOIN BY REGION
electricity_consumption_region <- read_csv("electricity_consumption_region.csv")

#filtering and getting the countries
electricity_consumption_region <-  electricity_consumption_region%>%
  filter(`Country or region`%in%c('England','Scotland','Wales'),
         `Local authority` == 'All local authorities')
#filtering just the local authorities

electricity_consumption_region <- electricity_consumption_region%>%
  filter(!grepl("inc",`Country or region`))
#grouping by region only
electricity_consumption_region <- electricity_consumption_region %>% group_by(`Country or region`,year)%>%
  summarise(consumption= mean(consumption,rm.na = T))

#getting the map object from countries using the the uk_json object
electricity_consumption_region <- electricity_consumption_region%>%
  left_join(uk_json,by = c('Country or region'='geo_label'))
electricity_consumption_region <- electricity_consumption_region%>%st_as_sf()

#CO2 EMISSIONS FUNCTION




# <----------------------------------------->
#CHARGE POINT DATA
#<----------------------------------------->

charge_point <- read_csv("national-charge-point-registry.csv")

 charge_point = charge_point%>%filter(dateCreated!='0000-00-00 00:00:00')%>%
  mutate(dateCreated = as_datetime( dateCreated),
         dateUpdated=as_datetime(dateUpdated),
         lastDateUpdated=as_datetime(lastUpdated))
#unique date %Y-01-01

charge_point = charge_point%>%
  mutate(dateCreated = as.Date(dateCreated),
         dateCreated = format(dateCreated,'%Y-01-01'))%>%
  rename('date' = 'dateCreated')%>%arrange(date)
#cumulative_data_plot charge point
new_by_year =charge_point%>%
  group_by(date)%>%
  summarise (new_by_year = n())%>%na.omit()


new_by_year$csum <- ave(new_by_year$new_by_year,FUN = cumsum)
charge_point_aggregated <- new_by_year%>%
  mutate(date = as.Date(date))

#selecting relevant variables
charge_point <- charge_point%>%
  select(chargeDeviceID,
         name,
```

```
         latitude,
         longitude,buildingName,
         street,date,postcode,
         deviceOwnerName,
         locationShortDescription)
charge_point = charge_point%>%
  filter(date != 'NA')



# <----------------------------------------->
#ELECTRICITY-VEHICLE
#<----------------------------------------->

electric_vehicle <- read_csv("ulev_clean.csv")
electric_vehicle%>%filter(Geography=='Great Britain',
                          `Date Interval [note 3]`=='Annual',
                          Units == 'Number',
                          `BodyType [note 2]`!='Total',
                          type!='Total')%>%
  group_by(date,`BodyType [note 2]`)%>%
  rename('Type of Vehicle'='BodyType [note 2]')%>%
  summarise(total = sum(count))%>%
  ggplot(aes(date, log(total),color =`Type of Vehicle` ))+
  geom_line()+
  geom_point()+
  ylab("Total Vehicle") + xlab("Date") + theme_bw()

#electri vehicle tab plot
ulev_clean_tab_plot <- read_csv("ulev_clean.csv")
ulev_clean_tab_plot <- ulev_clean_tab_plot%>%select(Geography,
                                                    `Date Interval [note 3]`,
                                                    Units,everything())%>%
  filter(type !='Total')
ulev_clean_tab_plot <- ulev_clean_tab_plot%>%
  rename('territory' ='Geography',
         'period'='Date Interval [note 3]',
         'types'='BodyType [note 2]')

ulev_clean_tab_plot <- ulev_clean_tab_plot %>%
  group_by(territory,
           period,
           date, types)%>%
  summarise(total = sum(count))

# <----------------------------------------->
#             uk_local_authority green house emission
#<----------------------------------------->
uk_local_authority <- read_csv("uk-local-authority-ghg-emissions-2020-dataset.csv")
#filtering co2 data
uk_local_authority <- uk_local_authority%>%filter(Country!='Unallocated',
                                                  `Greenhouse gas`=='CO2',
                                                  `LA GHG Sector`=='Transport')
uk_local_authority

#stadarizing the date
uk_local_authority <- uk_local_authority%>%
  mutate(`Calendar Year` = as.Date(paste(`Calendar Year`,'01','01',sep = '-')))%>%
  rename('date' = `Calendar Year`)%>%
  select(date,everything())


uk_local_authority_plot <- uk_local_authority%>%
  group_by(Country,date)%>%
  summarise(mean_co2=mean(`Territorial emissions (kt CO2e)`))
uk_local_authority_plot
#getting the aggregate table co2 for plot
#new cases by year
uk_local_authority_plot%>%
  ggplot(aes(date, mean_co2,color = Country))+
  geom_line()

#getting map object
uk_local_authority <- uk_local_authority_plot%>%
  left_join(uk_json,by = c('Country'='geo_label'))
uk_local_authority <- uk_local_authority%>%st_as_sf()

###############
# ELECTRICITY
#############
electricity <- read_excel('electricity_data.xlsx',na = c('-'))

 electricity_plot <- electricity%>%
pivot_longer(!ano)%>%
  filter(name != 'used_on_works')%>%
```

```
 mutate(ano = as.Date(paste(ano,'01','01',sep = '-')))%>%
   rename('date'='ano')



#electricity consumed
  electricity_tab_plot <- read_csv('electricity_consumption_region copia.csv')

  '%!in%' <- function(x,y)!('%in%'(x,y))
 electricity_tab_plot <-
  electricity_tab_plot%>%
    filter(`Local authority`!= 'All local authorities')%>%
    group_by(year,`Country or region`)%>%
    summarise(`Total consumption\r\n(GWh):\r\nAll meters`=sum(`Total consumption\r\n(GWh):\r\nAll meters`))%>%
    filter(`Country or region`%!in%c('Great Britain (unallocated)',
                                     'England and Wales (unallocated)',
                                     'Scotland (unallocated)'))%>%
    rename('date' = 'year',
           'territory'='Country or region',
           'consumption_Gwh'='Total consumption\r\n(GWh):\r\nAll meters')


  electricity_tab_plot$period <- 'Annual'

  electricity_tab_plot <- electricity_tab_plot%>%
    rename('total_consumption_Gwh'='consumption_Gwh')

  data=list('Electricity'=electricity_tab_plot,
            'Electric Vehicle'=ulev_clean_tab_plot)
#<------------------------------------------>
#                  MAPPING
#<------------------------------------------>
#coloring the map
bins=round(quantile(uk_local_authority$mean_co2, probs = seq(0, 1, 0.2)),0)%>%as.vector()
bins = c(bins,Inf)
bins
cv_pal <- colorBin("Blues", domain = uk_local_authority$mean_co2, bins = bins)
#
a <- '2024-01-01'



basemap = leaflet()%>%
  addTiles()%>%
  addLayersControl(
    position = 'bottomright',
    overlayGroups = c('location charge points', 'Co2 Emission'),
   options = layersControlOptions(collapsed=F))%>%
    addLegend(data = uk_local_authority,
              "bottomright", pal = cv_pal, values = ~uk_local_authority$mean_co2,
              title = "<small>Avarage Co2 emission (MTCO2 eq)</small>")
 # addMarkers(data = charge_point%>%filter(date<= a),
              #lat = ~latitude,
             # lng = ~longitude,
             # group = 'location charge points',
             # clusterOptions = markerClusterOptions())%>%
  #addPolygons(data = uk_local_authority%>%filter(date<=a),
              # group = 'Co2 Emission',
               #fillColor = ~cv_pal(uk_local_authority$mean_co2),
              # stroke = FALSE,
              # smoothFactor = 0.2,
              # fillOpacity = .1)
#SELECTING THE DATE
current_date = max(charge_point$date,na.rm = T)
choice = uk_local_authority$date%>%unique()
choice=c(choice,c('2021-01-01','2022-01-01'))
#CRATING THE SHINY MAP
ui <- bootstrapPage(
  tags$style("
        #controls {
          background-color: #fff;
          opacity: 0.5;
        }
        #controls:hover{
          opacity: 1;
        }
               "),
  navbarPage("UK Transport Zero-emission 2040",
             tabPanel("Map",
                      div(class="outer",
                          tags$head(includeCSS("estilos_.css")),

                      leafletOutput('mymap',
                                    width = '100%',
```

```
                              height = '100%'),
                    absolutePanel(id = "controls", class = "panel panel-default",
                              top = 75, left = 55, width = 450, fixed=TRUE,
                              draggable = TRUE, height = "auto",
                              plotOutput("cumulative_plot_cp",
                                         height="130px",
                                         width="100%"),
                              plotOutput("cumulative_plot_co2",
                                         height="230px",
                                         width="100%"),
                              plotOutput("Vehicles",
                                         height="230px",
                                         width="100%"),
                              plotOutput("Electricity",
                                         height="230px",
                                         width="100%"),
                              sliderTextInput("plot_date",
                                              label = h5("Select the date"),
                                              choices = choice[8:18],
                                              selected = '2022-01-01',
                                              grid = FALSE,
                                              animate=animationOptions(interval = 15000, loop = FALSE)),

                    )
          )
  ),
  tabPanel('Plots',
           selectInput('data', 'Data',
                       choices = names(data)%>%unique()),
           selectInput('territory', 'Territory', choices = NULL),
           selectInput('period', 'Period', choices = NULL),
           tableOutput('data'),
           plotOutput('date'))))

server <- function(input,output,session){
  formatted_date = reactive({
    format(as.Date(input$plot_date))
  })
  reactive_cp = reactive({
    charge_point%>% filter(date !='NA',
                           date <= formatted_date())
  })
  reactive_db_co2 = reactive({
    uk_local_authority%>% filter(
                                date <= formatted_date())
  })
  output$mymap <- renderLeaflet({
    basemap
  })
  observeEvent(input$plot_date,{
    leafletProxy('mymap') %>%
     clearMarkers()%>%
      clearMarkerClusters()%>%
      clearShapes()%>%
      addMarkers(data = reactive_cp(),
                 lat = ~latitude,
               lng = ~longitude,
                group = 'location charge points',
                 label = sprintf("<strong>Name:</strong> %s<br/>
                              <strong>Street</strong>: %s<br/>
                              <strong>Post Code</strong>:%s<br/>
                                 <strong>Owner</strong>:%s",
                                reactive_cp()$name,
                                 reactive_cp()$street,
               reactive_cp()$postcode,
               reactive_cp()$deviceOwnerName)%>%
                   lapply(htmltools::HTML),
                 clusterOptions = markerClusterOptions())%>%
     addPolygons(data =reactive_db_co2(),
                  group = 'Co2 Emission',
                 fillColor = ~cv_pal(reactive_db_co2()$mean_co2),
                 stroke = FALSE, smoothFactor = 0.2, fillOpacity =.1)



  })
  output$cumulative_plot_cp <- renderPlot(
    {
      charge_point_aggregated%>%
        filter(date<=formatted_date())%>%
        ggplot(aes(date,csum))+
        geom_line(color = 'blue') + geom_point(size = 1, alpha = 0.8,
                                               color = 'blue') +
        # geom_bar(position="stack", stat="identity") +
        ylab("Total Charge Points") + xlab("Date") + theme_bw() +
```

```
      ggtitle("Charge Points")
  })
output$cumulative_plot_co2 <- renderPlot(

  uk_local_authority_plot%>%filter(date<=formatted_date())%>%
    ggplot(aes(date, mean_co2,color = Country))+
    geom_line() + geom_point(size = 1, alpha = 0.8) +
    # geom_bar(position="stack", stat="identity") +
    ylab("Total Emission") + xlab("Date") + theme_bw() +
    ggtitle("Transport Total CO2 Emission (MTCO2 Eq.)")
)

output$Vehicles <- renderPlot(
  electric_vehicle%>%filter(Geography=='Great Britain',
                            `Date Interval [note 3]`=='Annual',
                            Units == 'Number',
                            `BodyType [note 2]`!='Total',
                            type!='Total')%>%
    group_by(date,`BodyType [note 2]`)%>%
    rename('Type of Vehicle'='BodyType [note 2]')%>%
    summarise(total = sum(count))%>%
    filter(date <=formatted_date())%>%
    ggplot(aes(date, log(total),color =`Type of Vehicle` ))+
    geom_line()+
    geom_point()+
    ylab("Total Vehicle") + xlab("Date") + theme_bw() +
    ggtitle('Total Vehicles (log)')
)
output$Electricity <- renderPlot(
  electricity_plot%>%
    filter(date<=formatted_date())%>%
    ggplot(aes(date, value, color = name))+
    geom_line()+
    geom_point()+
    theme_bw()+
    ggtitle('Electricity Generated By Source (Gwh)')
)

#Tab panel
data_<- reactive(
  data
)
select_data <- reactive(
  data_()[[input$data]]
)
#once we choose the data
observeEvent(select_data(),
             {
               choices <- unique(select_data()$territory)
               updateSelectInput(inputId = 'territory',
                                 choices = choices)
             })

territory_chosen <- reactive(
  select_data()%>%
    filter(territory == input$territory)
)

observeEvent(territory_chosen (),
             {
               choices <- unique(territory_chosen()$period)
               updateSelectInput(inputId = 'period',
                                 choices = choices)
             })




period_chosen <- reactive({
  territory_chosen()%>%
    filter(period == input$period)
}
)


output$data <-
  renderTable({
    req(input$period)
    head(period_chosen() %>%
           filter(period == input$period))
  })


plot<- reactive(
  if (input$data == 'Electricity'){
```

```
      period_chosen()%>%
        ggplot(aes(date,
                   total_consumption_Gwh))+
        geom_line(color = 'blue')+theme_bw()+
        ylab('Electricity Consumed (Gwh)')+
        xlab('date')+
        ggtitle('Electricity')+
        geom_point(color = 'blue')
    }else {
      period_chosen()%>%filter(types !='Total')%>%
        ggplot(aes(date,
                   log(total),color = types))+
        geom_line()+
        geom_point()+
        facet_wrap(~types,scales = 'free')+theme_bw()+
        ylab('Total electric Vehicle (log)')+
        xlab('date')+
        ggtitle('Electric Vehicles')
    })

  output$date <- renderPlot(plot())
}
shinyApp(ui,server)
```